

```
In [68]: from rdflib import Graph
from rdflib.namespace import RDF
import sys

def clean_ttl_file(input_file: str, output_file: str):
    # Load the RDF graph from the input file
    g = Graph()
    g.parse(input_file, format="turtle")

    # Create a new graph to write the cleaned version
    cleaned_graph = Graph()

    # Group all triples by subject and predicate
    grouped_triples = {}
    for s, p, o in g:
        grouped_triples.setdefault((s, p), set()).add(o)

    # Add grouped triples back into a new graph
    for (s, p), objects in grouped_triples.items():
        for o in objects:
            cleaned_graph.add((s, p, o))

    # Serialize to Turtle with cleaned output
    cleaned_graph.serialize(destination=output_file, format="turtle")

    print(f"Cleaned TTL written to {output_file}")

# Example usage
if __name__ == "__main__":
    input_ttl = "Pro_Go_gan_file.ttl" # Replace with your actual TTL file
    output_ttl = "Pro_Go_gan_cleaned.ttl"
    clean_ttl_file(input_ttl, output_ttl)
```

Cleaned TTL written to Pro_Go_gan_cleaned.ttl

```
In [77]: from rdflib import Graph, Namespace, URIRef, Literal
import pandas as pd

# Load your RDF file
g = Graph()
g.parse("Pro_Go_gan_Results.ttl", format="turtle")

# Define namespaces
RDF = Namespace("http://www.w3.org/1999/02/22-rdf-syntax-ns#")
oboInOwl = Namespace("http://www.geneontology.org/formats/oboInOwl#")
rdfs = Namespace("http://www.w3.org/2000/01/rdf-schema#")
owl = Namespace("http://www.w3.org/2002/07/owl#")
PR = Namespace("http://purl.obolibrary.org/obo/pr#")

# Bind namespaces for pretty QName output (optional)
g.bind("rdf", RDF)
g.bind("oboInOwl", oboInOwl)
g.bind("rdfs", rdfs)
g.bind("owl", owl)
g.bind("pr", PR)
```

```

# Old and new base URI strings for replacement
OLD_BASE = "http://example.org/default#"
NEW_BASE = "https://proconsortium.org/cgi-bin/entry_pro?id="

# Properties to extract
properties_to_extract = [
    owl.annotatedProperty,
    owl.annotatedSource,
    rdfs.comment,
    oboInOwl.hasBroadSynonym,
    oboInOwl.hasDbXref,
    oboInOwl.hasExactSynonym,
    oboInOwl.hasNarrowSynonym,
    oboInOwl.hasRelatedSynonym,
    rdfs.label,
    owl.onProperty,
    owl.someValuesFrom,
    rdfs.subClassOf,
    rdfs.type
]

data = []

# Loop over all protein sample subjects
for subj in g.subjects(predicate=RDF.type, object=None):
    subj_str = str(subj)
    if "PR_" in subj_str: # crude filter for protein samples
        # Replace old base URI with new OBO base URI
        if subj_str.startswith(OLD_BASE):
            protein_sample_uri = subj_str.replace(OLD_BASE, NEW_BASE)
        else:
            protein_sample_uri = subj_str

    entry = {"protein_sample": protein_sample_uri}

    # Extract properties
    for prop in properties_to_extract:
        values = [str(o) for o in g.objects(subj, prop)]

        # Flatten comma-separated string literals if any
        flat = []
        for v in values:
            if isinstance(v, str) and "," in v:
                flat.extend([item.strip() for item in v.split(",")])
            else:
                flat.append(v.strip())

        entry[str(prop)] = flat
        entry[str(prop) + "_count"] = len(flat)

    data.append(entry)

# Create DataFrame
df = pd.DataFrame(data)

```

```
# Optional: show all columns  
pd.set_option('display.max_columns', None)  
print(df.head())
```

	protein_sample \
0	https://proconsortium.org/cgi-bin/entry_pro?id...
1	https://proconsortium.org/cgi-bin/entry_pro?id...
2	https://proconsortium.org/cgi-bin/entry_pro?id...
3	https://proconsortium.org/cgi-bin/entry_pro?id...
4	https://proconsortium.org/cgi-bin/entry_pro?id...

	http://www.w3.org/2002/07/owl#annotatedProperty \
0	[]
1	[]
2	[]
3	[]
4	[]

	http://www.w3.org/2002/07/owl#annotatedProperty_count \
0	0
1	0
2	0
3	0
4	0

	http://www.w3.org/2002/07/owl#annotatedSource \
0	[]
1	[]
2	[]
3	[]
4	[]

	http://www.w3.org/2002/07/owl#annotatedSource_count \
0	0
1	0
2	0
3	0
4	0

	http://www.w3.org/2000/01/rdf-schema#comment \
0	[]
1	[]
2	[]
3	[]
4	[]

	http://www.w3.org/2000/01/rdf-schema#comment_count \
0	0
1	0
2	0
3	0
4	0

	http://www.geneontology.org/formats/oboInOwl##hasBroadSynonym \
0	[]
1	[]
2	[]
3	[]
4	[]

	http://www.geneontology.org/formats/oboInOwl##hasBroadSynonym_count \
0	0
1	0
2	0
3	0
4	0

	http://www.geneontology.org/formats/oboInOwl##hasDbXref \
0	[]
1	[]
2	[]
3	[]
4	[]

	http://www.geneontology.org/formats/oboInOwl##hasDbXref_count \
0	0
1	0
2	0
3	0
4	0

	http://www.geneontology.org/formats/oboInOwl##hasExactSynonym \
0	[]
1	[]
2	[]
3	[]
4	[]

	http://www.geneontology.org/formats/oboInOwl##hasExactSynonym_count \
0	0
1	0
2	0
3	0
4	0

	http://www.geneontology.org/formats/oboInOwl##hasNarrowSynonym \
0	[]
1	[]
2	[]
3	[]
4	[]

	http://www.geneontology.org/formats/oboInOwl##hasNarrowSynonym_count \
0	0
1	0
2	0
3	0
4	0

	http://www.geneontology.org/formats/oboInOwl##hasRelatedSynonym \
0	[]
1	[]
2	[]
3	[]
4	[]

	http://www.geneontology.org/formats/oboInOwl##hasRelatedSynonym_count \
0	0
1	0
2	0
3	0
4	0

	http://www.w3.org/2000/01/rdf-schema#label \
0	[]
1	[]
2	[]
3	[]
4	[]

	http://www.w3.org/2000/01/rdf-schema#label_count \
0	0
1	0
2	0
3	0
4	0

	http://www.w3.org/2002/07/owl#onProperty \
0	[]
1	[]
2	[]
3	[]
4	[]

	http://www.w3.org/2002/07/owl#onProperty_count \
0	0
1	0
2	0
3	0
4	0

	http://www.w3.org/2002/07/owl#someValuesFrom \
0	[]
1	[]
2	[]
3	[]
4	[]

	http://www.w3.org/2002/07/owl#someValuesFrom_count \
0	0
1	0
2	0
3	0
4	0

	http://www.w3.org/2000/01/rdf-schema#subClassOf \
0	[]
1	[]
2	[]
3	[]
4	[]

	http://www.w3.org/2000/01/rdf-schema#subClassOf_count \
0	0
1	0
2	0
3	0
4	0

	http://www.w3.org/2000/01/rdf-schema#type \
0	[]
1	[]
2	[]
3	[]
4	[]

	http://www.w3.org/2000/01/rdf-schema#type_count
0	0
1	0
2	0
3	0
4	0

```
In [71]: import re

# Extract base ID from full sample URI
def extract_protein_base_id(uri):
    match = re.search(r"PR_\d+", uri)
    return match.group(0) if match else None

# Add the base ID to the DataFrame
df["protein_base_id"] = df["protein_sample"].apply(extract_protein_base_id)
```

```
In [72]: grouped = df.groupby("protein_base_id")
```

```
In [73]: print(grouped.head())
```

```

                                protein_sample \
0      http://example.org/default#PR_000000453_1
1      http://example.org/default#PR_000000453_2
2      http://example.org/default#PR_000000453_3
3      http://example.org/default#PR_000000453_4
4      http://example.org/default#PR_000000453_5
...
1945   http://example.org/default#PR_000085667_1
1946   http://example.org/default#PR_000085667_2
1947   http://example.org/default#PR_000085667_3
1948   http://example.org/default#PR_000085667_4
1949   http://example.org/default#PR_000085667_5

```

```

                                http://www.w3.org/2002/07/owl#annotatedProperty \
0      []
1      []
2      []
3      []
4      []
...
1945   []
1946   []
1947   []
1948   []
1949   []

```

```

                                http://www.w3.org/2002/07/owl#annotatedProperty_count \
0      0
1      0
2      0
3      0
4      0
...
1945   0
1946   0
1947   0
1948   0
1949   0

```

```

                                http://www.w3.org/2002/07/owl#annotatedSource \
0      []
1      []
2      []
3      []
4      []
...
1945   []
1946   []
1947   []
1948   []
1949   []

```

```

                                http://www.w3.org/2002/07/owl#annotatedSource_count \
0      0
1      0
2      0

```


3	0
4	0
...	...
1945	0
1946	0
1947	0
1948	0
1949	0

http://www.w3.org/2000/01/rdf-schema#comment \	
0	[]
1	[]
2	[]
3	[]
4	[]
...	...
1945	[]
1946	[]
1947	[]
1948	[]
1949	[]

http://www.w3.org/2000/01/rdf-schema#comment_count \	
0	0
1	0
2	0
3	0
4	0
...	...
1945	0
1946	0
1947	0
1948	0
1949	0

http://www.geneontology.org/formats/oboInOwl##hasBroadSynonym \	
0	[]
1	[]
2	[]
3	[]
4	[]
...	...
1945	[]
1946	[]
1947	[]
1948	[]
1949	[]

http://www.geneontology.org/formats/oboInOwl##hasBroadSynonym_count \	
0	0
1	0
2	0
3	0
4	0
...	...
1945	0

1946	0
1947	0
1948	0
1949	0

	http://www.geneontology.org/formats/oboInOwl##hasDbXref \
0	[]
1	[]
2	[]
3	[]
4	[]
...	...
1945	[]
1946	[]
1947	[]
1948	[]
1949	[]

	http://www.geneontology.org/formats/oboInOwl##hasDbXref_count \
0	0
1	0
2	0
3	0
4	0
...	...
1945	0
1946	0
1947	0
1948	0
1949	0

	http://www.geneontology.org/formats/oboInOwl##hasExactSynonym \
0	[]
1	[]
2	[]
3	[]
4	[]
...	...
1945	[]
1946	[]
1947	[]
1948	[]
1949	[]

	http://www.geneontology.org/formats/oboInOwl##hasExactSynonym_count \
0	0
1	0
2	0
3	0
4	0
...	...
1945	0
1946	0
1947	0
1948	0
1949	0

	http://www.geneontology.org/formats/oboInOwl##hasNarrowSynonym	\
0		[]
1		[]
2		[]
3		[]
4		[]
...		...
1945		[]
1946		[]
1947		[]
1948		[]
1949		[]

	http://www.geneontology.org/formats/oboInOwl##hasNarrowSynonym_count
\	
0	0
1	0
2	0
3	0
4	0
...	...
1945	0
1946	0
1947	0
1948	0
1949	0

	http://www.geneontology.org/formats/oboInOwl##hasRelatedSynonym	\
0		[]
1		[]
2		[]
3		[]
4		[]
...		...
1945		[]
1946		[]
1947		[]
1948		[]
1949		[]

	http://www.geneontology.org/formats/oboInOwl##hasRelatedSynonym_count
\	
0	0
1	0
2	0
3	0
4	0
...	...
1945	0
1946	0
1947	0
1948	0
1949	0

<http://www.w3.org/2000/01/rdf-schema#label> \

0	[]
1	[]
2	[]
3	[]
4	[]
...	...
1945	[]
1946	[]
1947	[]
1948	[]
1949	[]

http://www.w3.org/2000/01/rdf-schema#label_count \	
0	0
1	0
2	0
3	0
4	0
...	...
1945	0
1946	0
1947	0
1948	0
1949	0

http://www.w3.org/2002/07/owl#onProperty \	
0	[]
1	[]
2	[]
3	[]
4	[]
...	...
1945	[]
1946	[]
1947	[]
1948	[]
1949	[]

http://www.w3.org/2002/07/owl#onProperty_count \	
0	0
1	0
2	0
3	0
4	0
...	...
1945	0
1946	0
1947	0
1948	0
1949	0

http://www.w3.org/2002/07/owl#someValuesFrom \	
0	[]
1	[]
2	[]
3	[]

4	[]
...	...
1945	[]
1946	[]
1947	[]
1948	[]
1949	[]

	http://www.w3.org/2002/07/owl#someValuesFrom_count \
0	0
1	0
2	0
3	0
4	0
...	...
1945	0
1946	0
1947	0
1948	0
1949	0

	http://www.w3.org/2000/01/rdf-schema#subClassOf \
0	[]
1	[]
2	[]
3	[]
4	[]
...	...
1945	[]
1946	[]
1947	[]
1948	[]
1949	[]

	http://www.w3.org/2000/01/rdf-schema#subClassOf_count \
0	0
1	0
2	0
3	0
4	0
...	...
1945	0
1946	0
1947	0
1948	0
1949	0

	http://www.w3.org/2000/01/rdf-schema#type \
0	[]
1	[]
2	[]
3	[]
4	[]
...	...
1945	[]
1946	[]

1947		PR_000000453
1948		PR_000000453
1949		PR_000000453
	http://www.w3.org/2000/01/rdf-schema#type_count	protein_base_id
0	0	PR_000000453
1	0	PR_000000453
2	0	PR_000000453
3	0	PR_000000453
4	0	PR_000000453
...	...	...
1945	0	PR_000085667
1946	0	PR_000085667
1947	0	PR_000085667
1948	0	PR_000085667
1949	0	PR_000085667

[1950 rows x 28 columns]

```
In [74]: import pandas as pd

properties_to_extract = [
    owl.annotatedProperty,
    owl.annotatedSource,
    rdfs.comment,
    oboInOwl.hasBroadSynonym,
    oboInOwl.hasDbXref,
    oboInOwl.hasExactSynonym,
    oboInOwl.hasNarrowSynonym,
    oboInOwl.hasRelatedSynonym,
    rdfs.label,
    owl.onProperty,
    owl.someValuesFrom,
    rdfs.subClassOf,
    rdfs.type
]

# Build aggregation dict dynamically for each property URI string with the "
agg_dict = {str(prop) + '_count': ['mean', 'std', 'min', 'max'] for prop in

# Then aggregate
summary = grouped.agg(agg_dict)

print(summary.head())
```

http://www.w3.org/2002/07/owl#annotatedProperty_count		
\	mean	std
protein_base_id		
PR_000000453	0.0	0.0
PR_000000843	0.0	0.0
PR_000000992	0.0	0.0
PR_000001167	0.0	0.0
PR_000001705	0.0	0.0

http://www.w3.org/2002/07/owl#annotatedSource_count			
\	min	max	mean
protein_base_id			
PR_000000453	0	0	0.0
PR_000000843	0	0	0.0
PR_000000992	0	0	0.0
PR_000001167	0	0	0.0
PR_000001705	0	0	0.0

\			
protein_base_id	std	min	max
PR_000000453	0.0	0	0
PR_000000843	0.0	0	0
PR_000000992	0.0	0	0
PR_000001167	0.0	0	0
PR_000001705	0.0	0	0

http://www.w3.org/2000/01/rdf-schema#comment_count			
\	mean	std	min
protein_base_id			
PR_000000453	0.0	0.0	0
PR_000000843	0.0	0.0	0
PR_000000992	0.0	0.0	0
PR_000001167	0.0	0.0	0
PR_000001705	0.0	0.0	0

\	
protein_base_id	max
PR_000000453	0
PR_000000843	0
PR_000000992	0
PR_000001167	0
PR_000001705	0

http://www.geneontology.org/formats/oboInOwl##hasBroadSynonym_count	
m_count	\
mean	
protein_base_id	
PR_000000453	0.0
PR_000000843	0.0
PR_000000992	0.0

PR_0000001167	0.0
PR_0000001705	0.0

	std	min	max
protein_base_id			
PR_0000000453	0.0	0	0
PR_0000000843	0.0	0	0
PR_0000000992	0.0	0	0
PR_0000001167	0.0	0	0
PR_0000001705	0.0	0	0

http://www.geneontology.org/formats/oboInOwl###hasDbXref_count

t \

mean

n	
protein_base_id	
PR_0000000453	0.0
PR_0000000843	0.0
PR_0000000992	0.0
PR_0000001167	0.0
PR_0000001705	0.0

	std	min	max
protein_base_id			
PR_0000000453	0.0	0	0
PR_0000000843	0.0	0	0
PR_0000000992	0.0	0	0
PR_0000001167	0.0	0	0
PR_0000001705	0.0	0	0

<http://www.geneontology.org/formats/oboInOwl###hasExactSynonym>

m_count \

mean

protein_base_id	
PR_0000000453	0.0
PR_0000000843	0.0
PR_0000000992	0.0
PR_0000001167	0.0
PR_0000001705	0.0

	std	min	max
protein_base_id			
PR_0000000453	0.0	0	0
PR_0000000843	0.0	0	0
PR_0000000992	0.0	0	0
PR_0000001167	0.0	0	0
PR_0000001705	0.0	0	0

<http://www.geneontology.org/formats/oboInOwl###hasNarrowSynonym>

ym_count \

mean

protein_base_id	
PR_000000453	0.0
PR_000000843	0.0
PR_000000992	0.0
PR_000001167	0.0
PR_000001705	0.0

	std	min	max
protein_base_id			
PR_000000453	0.0	0	0
PR_000000843	0.0	0	0
PR_000000992	0.0	0	0
PR_000001167	0.0	0	0
PR_000001705	0.0	0	0

<http://www.geneontology.org/formats/oboInOwl##hasRelatedSynonym>
 nym_count \

mean	
protein_base_id	
PR_000000453	0.0
PR_000000843	0.0
PR_000000992	0.0
PR_000001167	0.0
PR_000001705	0.0

http://www.w3.org/2000/01/rdf-schema#label_count
 t \

	std	min	max	mean
n				
protein_base_id				
PR_000000453	0.0	0	0	0.
0				
PR_000000843	0.0	0	0	0.
0				
PR_000000992	0.0	0	0	0.
0				
PR_000001167	0.0	0	0	0.
0				
PR_000001705	0.0	0	0	0.
0				

http://www.w3.org/2002/07/owl#onProperty_count
 \

	std	min	max	mean
protein_base_id				
PR_000000453	0.0	0	0	0.0
PR_000000843	0.0	0	0	0.0
PR_000000992	0.0	0	0	0.0
PR_000001167	0.0	0	0	0.0
PR_000001705	0.0	0	0	0.0

\

	std	min	max
protein_base_id			

PR_000000453	0.0	0	0
PR_000000843	0.0	0	0
PR_000000992	0.0	0	0
PR_000001167	0.0	0	0
PR_000001705	0.0	0	0

http://www.w3.org/2002/07/owl#someValuesFrom_count

\

	mean	std	min
protein_base_id			
PR_000000453	0.0	0.0	0
PR_000000843	0.0	0.0	0
PR_000000992	0.0	0.0	0
PR_000001167	0.0	0.0	0
PR_000001705	0.0	0.0	0

http://www.w3.org/2000/01/rdf-schema#subClassOf_count \

	max	mean
protein_base_id		
PR_000000453	0	0.0
PR_000000843	0	0.0
PR_000000992	0	0.0
PR_000001167	0	0.0
PR_000001705	0	0.0

http://www.w3.org/2000/01/rdf-schema#type_count

\

	std	min	max	mean
protein_base_id				
PR_000000453	0.0	0	0	0.0
PR_000000843	0.0	0	0	0.0
PR_000000992	0.0	0	0	0.0
PR_000001167	0.0	0	0	0.0
PR_000001705	0.0	0	0	0.0

	std	min	max
protein_base_id			
PR_000000453	0.0	0	0
PR_000000843	0.0	0	0
PR_000000992	0.0	0	0
PR_000001167	0.0	0	0
PR_000001705	0.0	0	0

```
In [75]: # Flatten multi-index columns created by agg()
summary.columns = ['_'.join(col).strip() for col in summary.columns.values]

# Reset index for easier plotting and handling
summary = summary.reset_index()

# Melt the dataframe to long format focusing on 'mean' statistics
melted = summary.melt(
    id_vars=['protein_base_id'], # keep the group identifier column(s)
    value_vars=[col for col in summary.columns if '_mean' in col], # only n
    var_name='property_stat',
    value_name='value'
```

```
)

# Extract property name only (remove the trailing '_mean')
melted['property'] = melted['property_stat'].apply(lambda x: x.replace('_mean', ''))

print(melted.head())
```

	protein_base_id	property_stat	value
0	PR_000000453	http://www.w3.org/2002/07/owl#annotatedPropert...	0.0
1	PR_000000843	http://www.w3.org/2002/07/owl#annotatedPropert...	0.0
2	PR_000000992	http://www.w3.org/2002/07/owl#annotatedPropert...	0.0
3	PR_000001167	http://www.w3.org/2002/07/owl#annotatedPropert...	0.0
4	PR_000001705	http://www.w3.org/2002/07/owl#annotatedPropert...	0.0

	property
0	http://www.w3.org/2002/07/owl#annotatedPropert...
1	http://www.w3.org/2002/07/owl#annotatedPropert...
2	http://www.w3.org/2002/07/owl#annotatedPropert...
3	http://www.w3.org/2002/07/owl#annotatedPropert...
4	http://www.w3.org/2002/07/owl#annotatedPropert...

In [66]: `import plotly.express as px`

```
# Encode protein_base_id as categorical numeric for X-axis
melted['protein_num'] = melted['protein_base_id'].astype('category').cat.codes

# Encode property as categorical numeric for Y-axis
melted['property_num'] = melted['property'].astype('category').cat.codes

fig = px.scatter_3d(
    melted,
    x='protein_num',
    y='property_num',
    z='value',
    color='protein_base_id',      # Color by protein_base_id
    symbol='property',           # Different symbol for each property
    labels={
        'protein_num': 'Protein Base ID',
        'property_num': 'Property',
        'value': 'Mean Count'
    },
    title='3D Distribution of Property Counts per Protein'
)

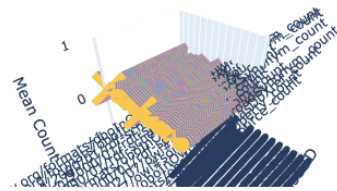
# Update axis ticks to show categorical labels instead of numeric codes
fig.update_layout(
    scene=dict(
        xaxis=dict(
            tickmode='array',
            tickvals=melted['protein_num'].unique(),
            ticktext=melted['protein_base_id'].astype('category').cat.categories
        ),
        yaxis=dict(
            tickmode='array',
            tickvals=melted['property_num'].unique(),
```

```

        ticktext=melted['property'].astype('category').cat.categories
    )
)
fig.show()

```

3D Distribution of Property Counts per Protein



protein_base_id, property

- PR_000000453, http://www.w3.org/2002/07/owl#annotatedProperty_count
- ◆ PR_000000453, http://www.w3.org/2002/07/owl#annotatedSource_count
- PR_000000453, http://www.w3.org/2000/01/rdf-schema#comment_count
- ✱ PR_000000453, http://www.geneontology.org/formats/oboInOwl#hasBroadSynonym_count
- ✦ PR_000000453, http://www.geneontology.org/formats/oboInOwl#hasDbXref_count
- PR_000000453, http://www.geneontology.org/formats/oboInOwl#hasExactSynonym_count
- ◆ PR_000000453, http://www.geneontology.org/formats/oboInOwl#hasNarrowSynonym_count
- PR_000000453, http://www.geneontology.org/formats/oboInOwl#hasRelatedSynonym_count

```

In [76]: import matplotlib.pyplot as plt
import seaborn as sns
from collections import Counter

# List of property names as strings (keys in df)
property_keys = [str(p) for p in properties_to_extract]

# For each property, aggregate all values across samples
for prop in property_keys:
    # Extract all lists of values for this property in all rows
    all_values_lists = df[prop].dropna().tolist()

    # Flatten list of lists into one big list of values
    all_values = [val for sublist in all_values_lists for val in sublist]

    if len(all_values) == 0:
        print(f"No data for property {prop}")
        continue

    # Count frequency of each distinct value
    value_counts = Counter(all_values)

    # Convert to DataFrame for plotting
    count_df = pd.DataFrame(value_counts.items(), columns=['Value', 'Count'])
    count_df = count_df.sort_values('Count', ascending=False).head(20) # To

    # Plot
    plt.figure(figsize=(10,6))
    sns.barplot(data=count_df, x='Count', y='Value', palette='viridis')
    plt.title(f'Distribution of values for property:\n{prop}')
    plt.xlabel('Count')
    plt.ylabel('Value')
    plt.tight_layout()
    plt.show()

```

No data for property <http://www.w3.org/2002/07/owl#annotatedProperty>
 No data for property <http://www.w3.org/2002/07/owl#annotatedSource>
 No data for property <http://www.w3.org/2000/01/rdf-schema#comment>
 No data for property <http://www.geneontology.org/formats/oboInOwl##hasBroadSynonym>
 No data for property <http://www.geneontology.org/formats/oboInOwl##hasDbXref>
 No data for property <http://www.geneontology.org/formats/oboInOwl##hasExactSynonym>
 No data for property <http://www.geneontology.org/formats/oboInOwl##hasNarrowSynonym>
 No data for property <http://www.geneontology.org/formats/oboInOwl##hasRelatedSynonym>
 No data for property <http://www.w3.org/2000/01/rdf-schema#label>
 No data for property <http://www.w3.org/2002/07/owl#onProperty>
 No data for property <http://www.w3.org/2002/07/owl#someValuesFrom>
 No data for property <http://www.w3.org/2000/01/rdf-schema#subClassOf>
 No data for property <http://www.w3.org/2000/01/rdf-schema#type>

```
In [23]: import altair as alt
import pandas as pd
from collections import Counter

alt.renderers.enable('default')

property_keys = [str(p) for p in properties_to_extract]

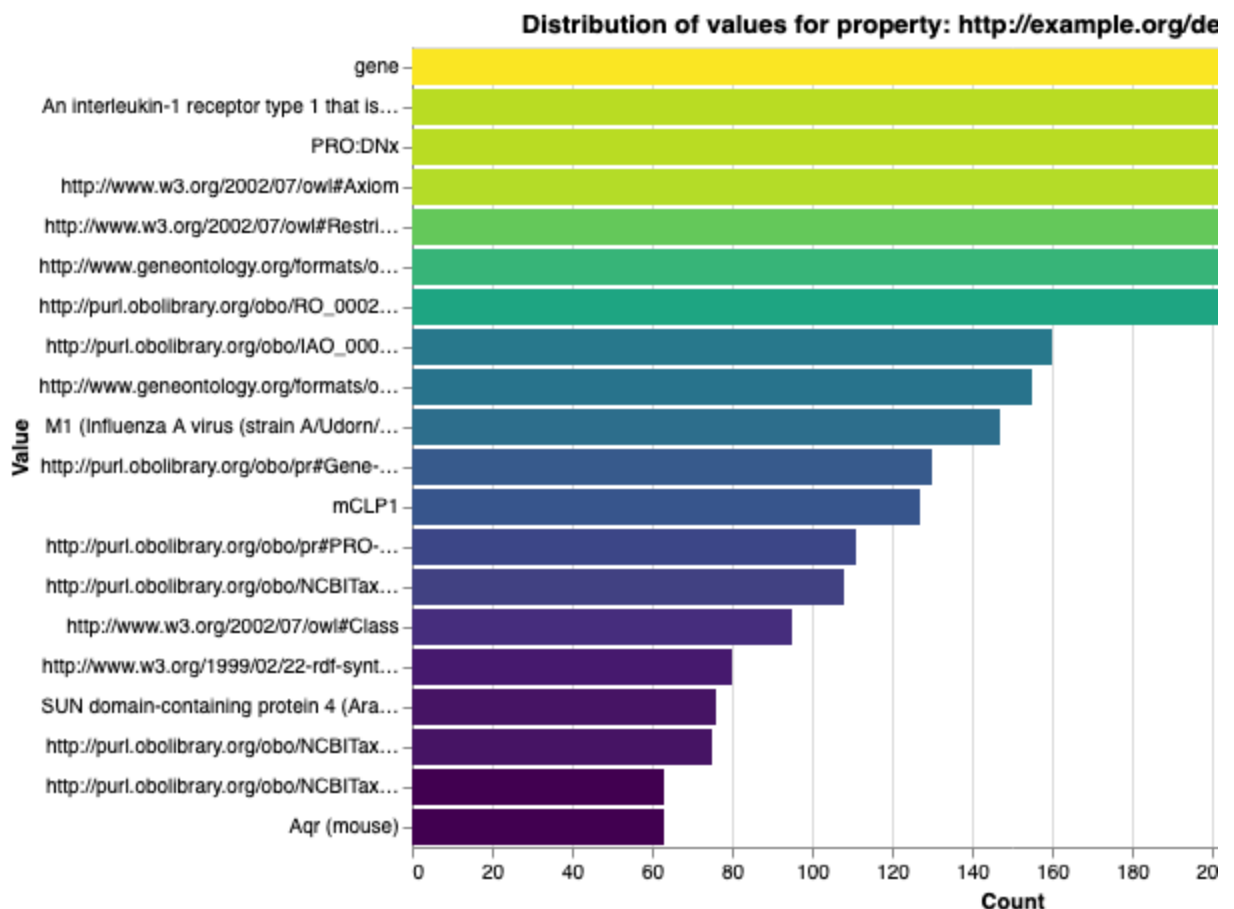
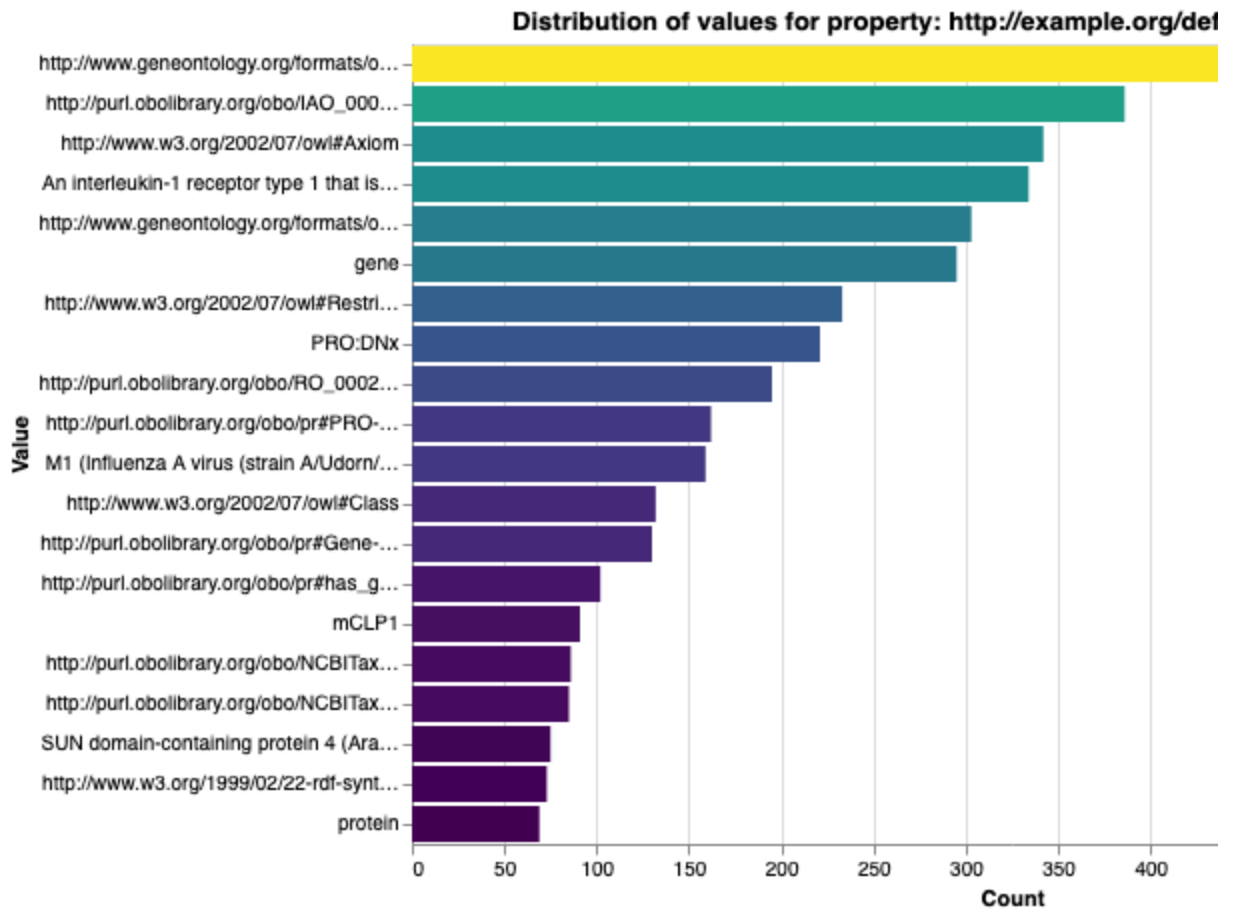
for prop in property_keys:
    all_values_lists = df[prop].dropna().tolist()
    all_values = [val for sublist in all_values_lists for val in sublist]

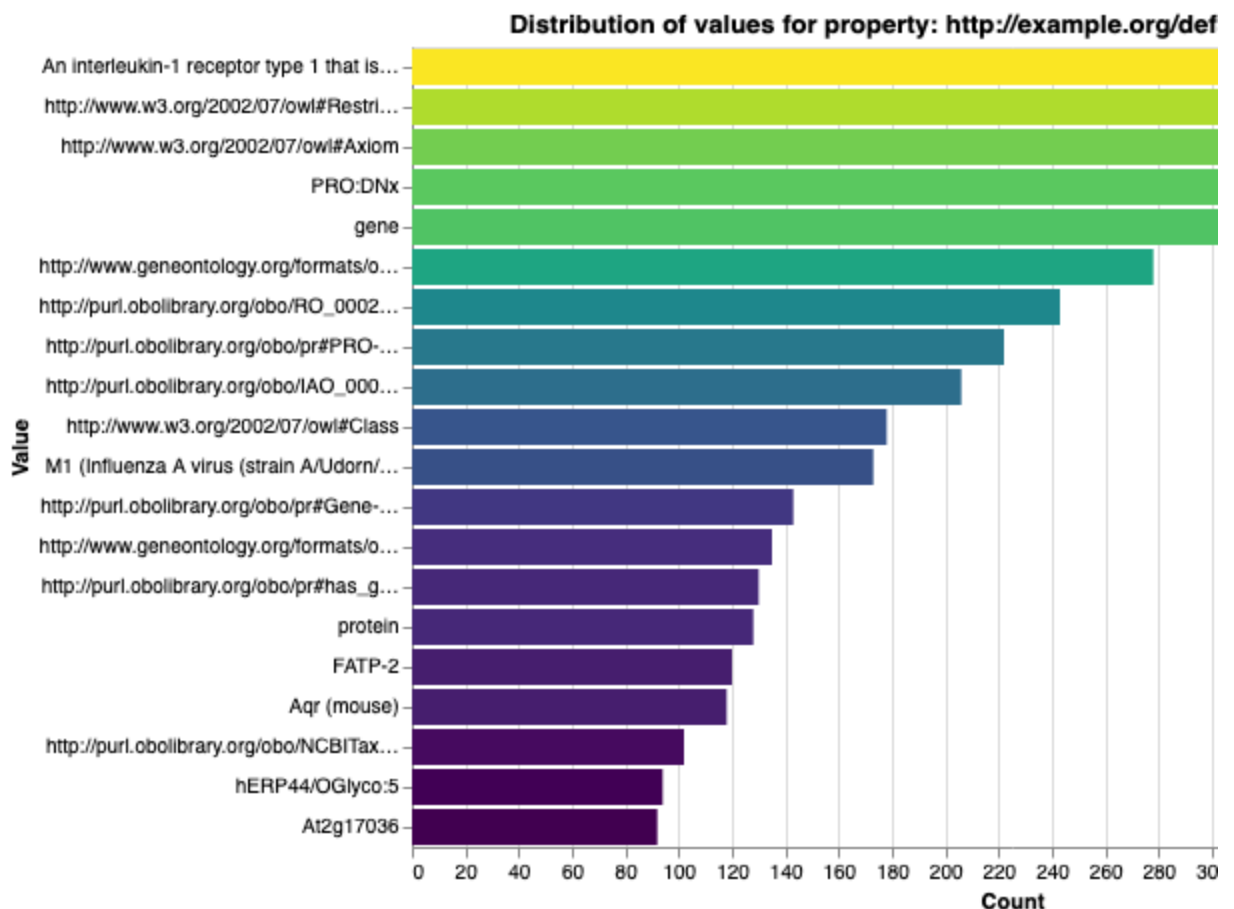
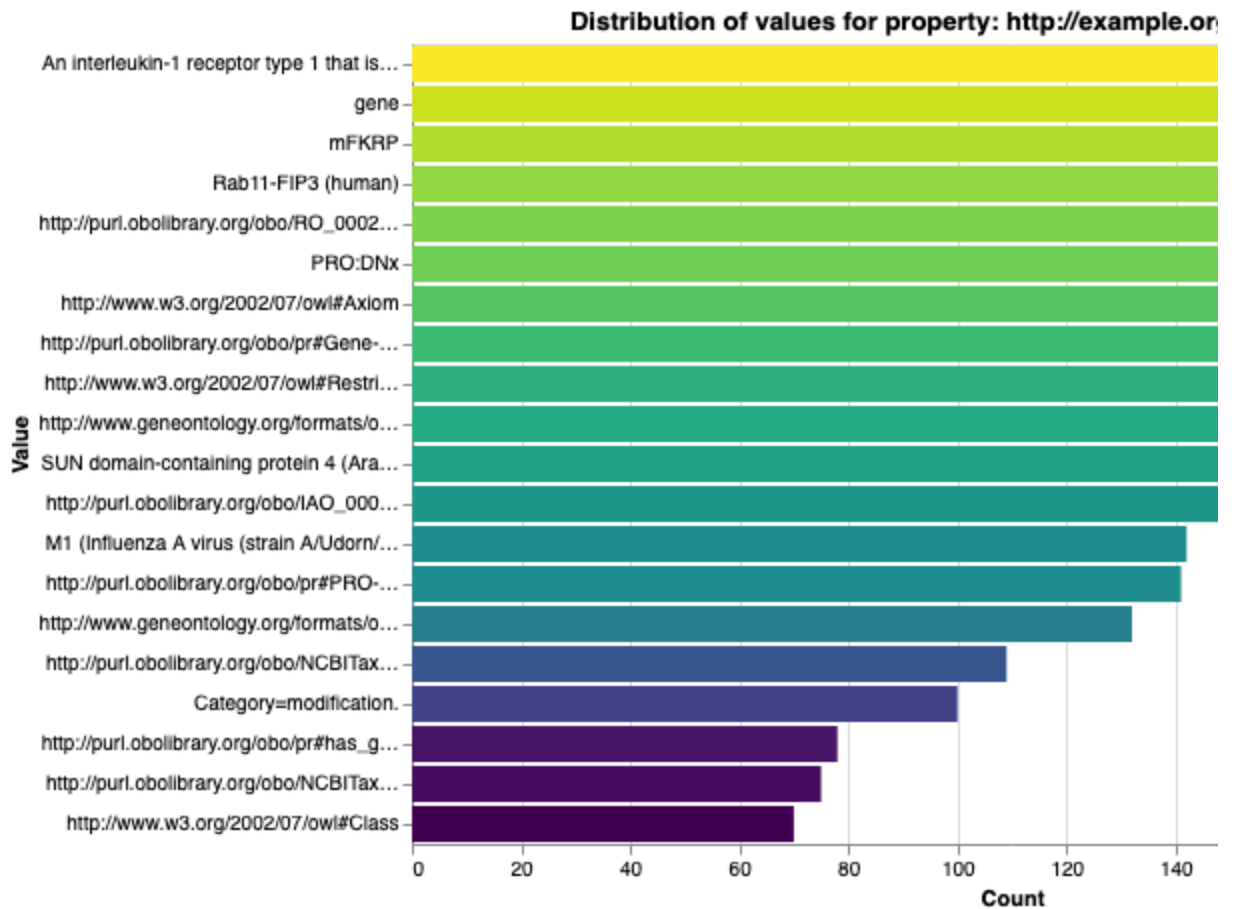
    if len(all_values) == 0:
        print(f"No data for property {prop}")
        continue

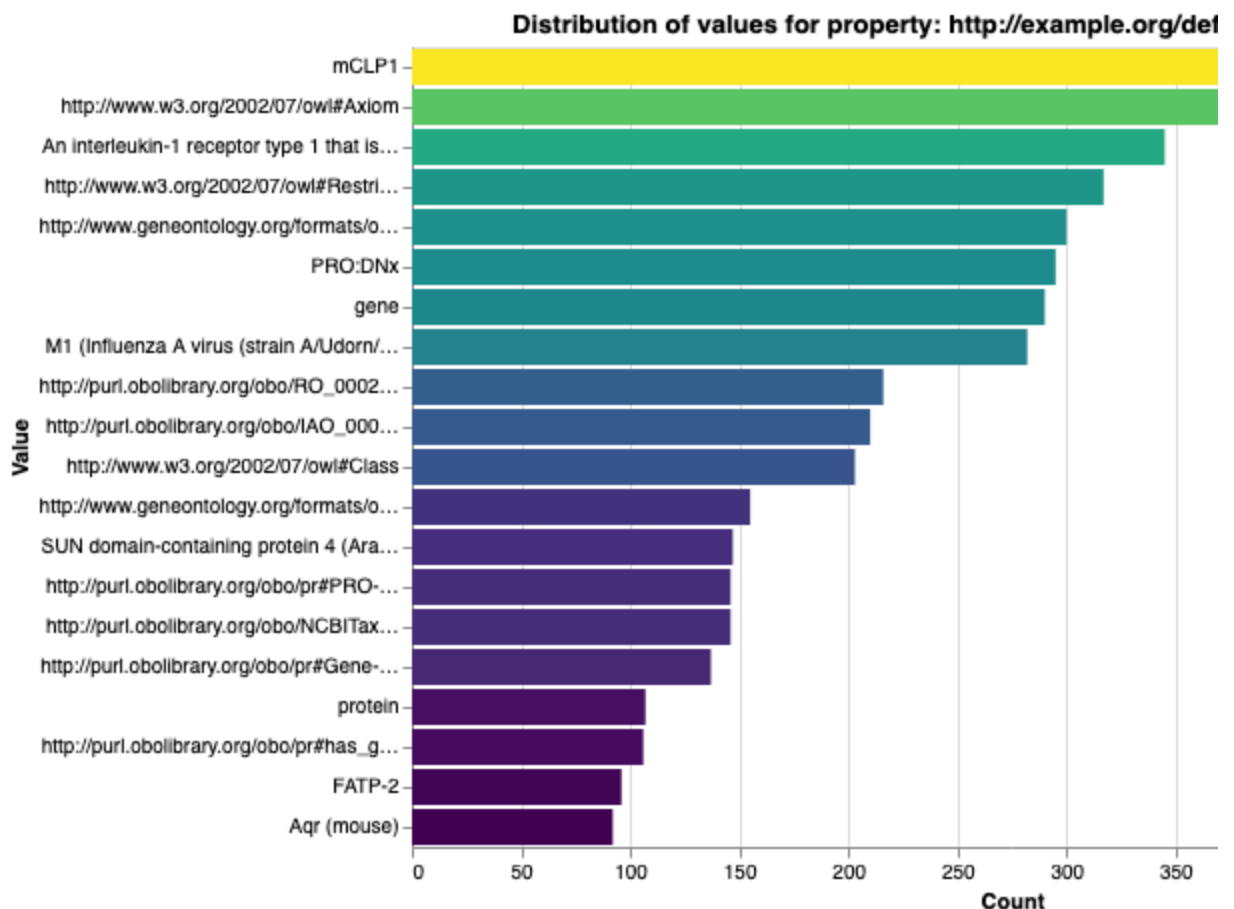
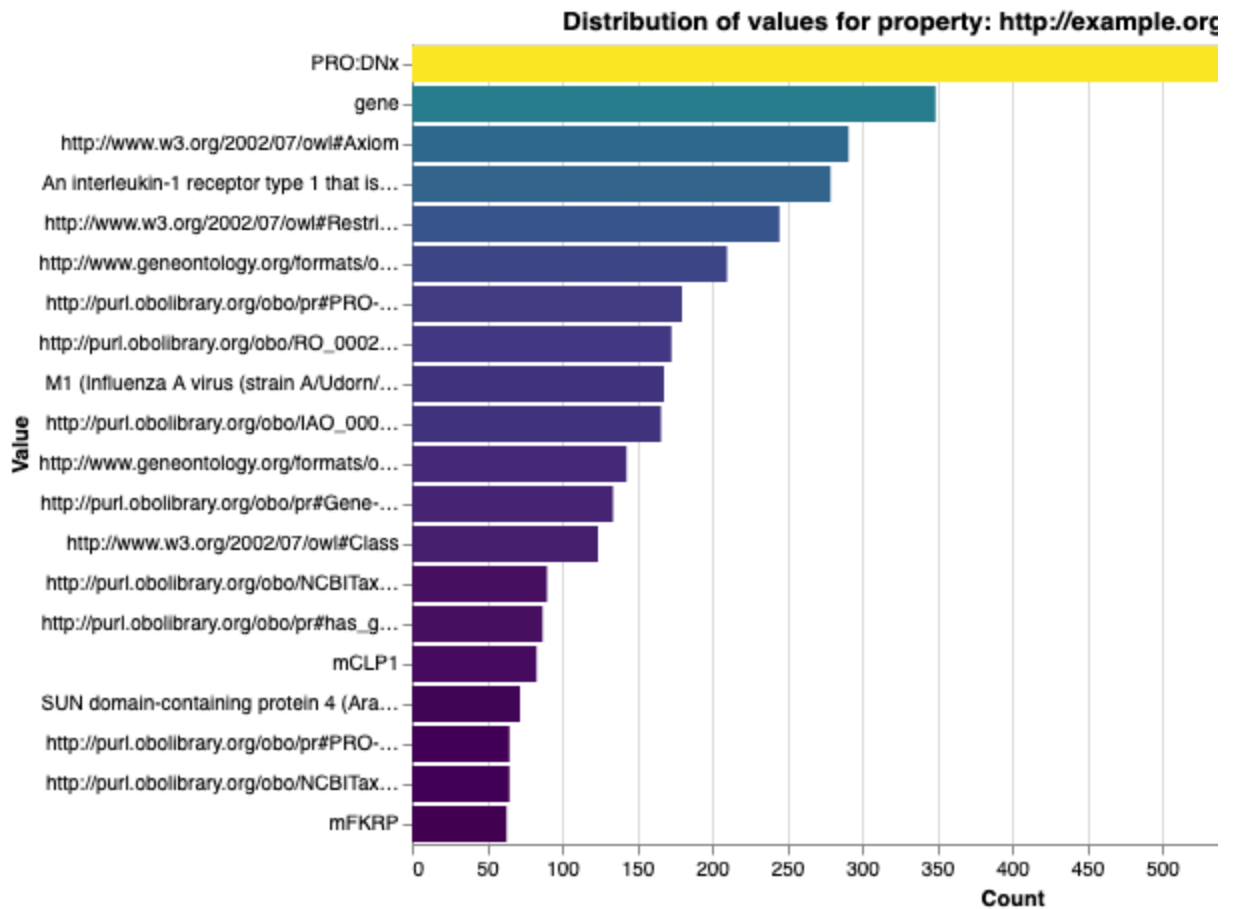
    value_counts = Counter(all_values)
    count_df = pd.DataFrame(value_counts.items(), columns=['Value', 'Count'])
    count_df = count_df.sort_values('Count', ascending=False).head(20) # to

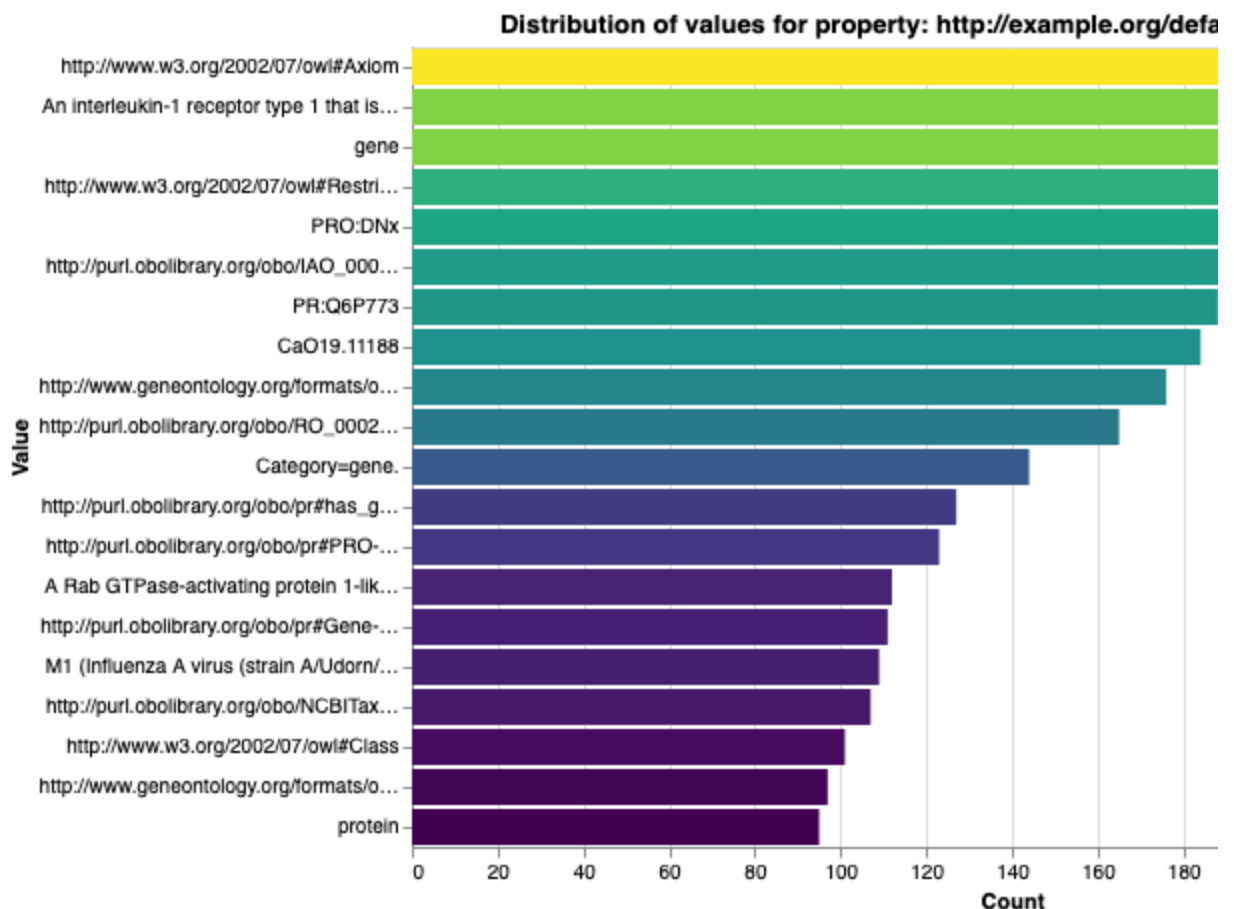
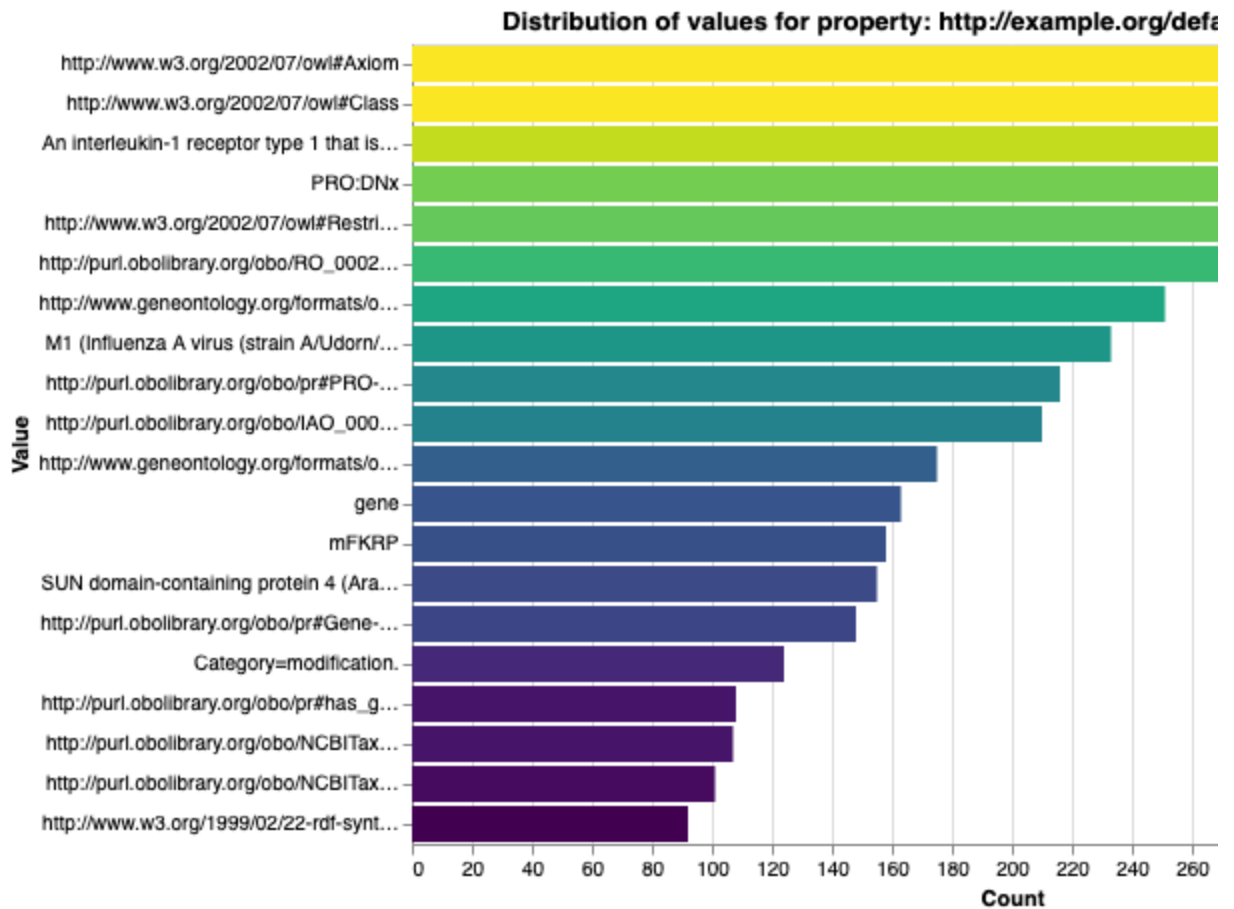
    chart = alt.Chart(count_df).mark_bar().encode(
        y=alt.Y('Value:N', sort='-x', title='Value'),
        x=alt.X('Count:Q', title='Count'),
        color=alt.Color('Count:Q', scale=alt.Scale(scheme='viridis'), legend=
        tooltip=['Value', 'Count'])
    ).properties(
        title=f'Distribution of values for property:\n{prop}',
        width=600,
        height=400
    )

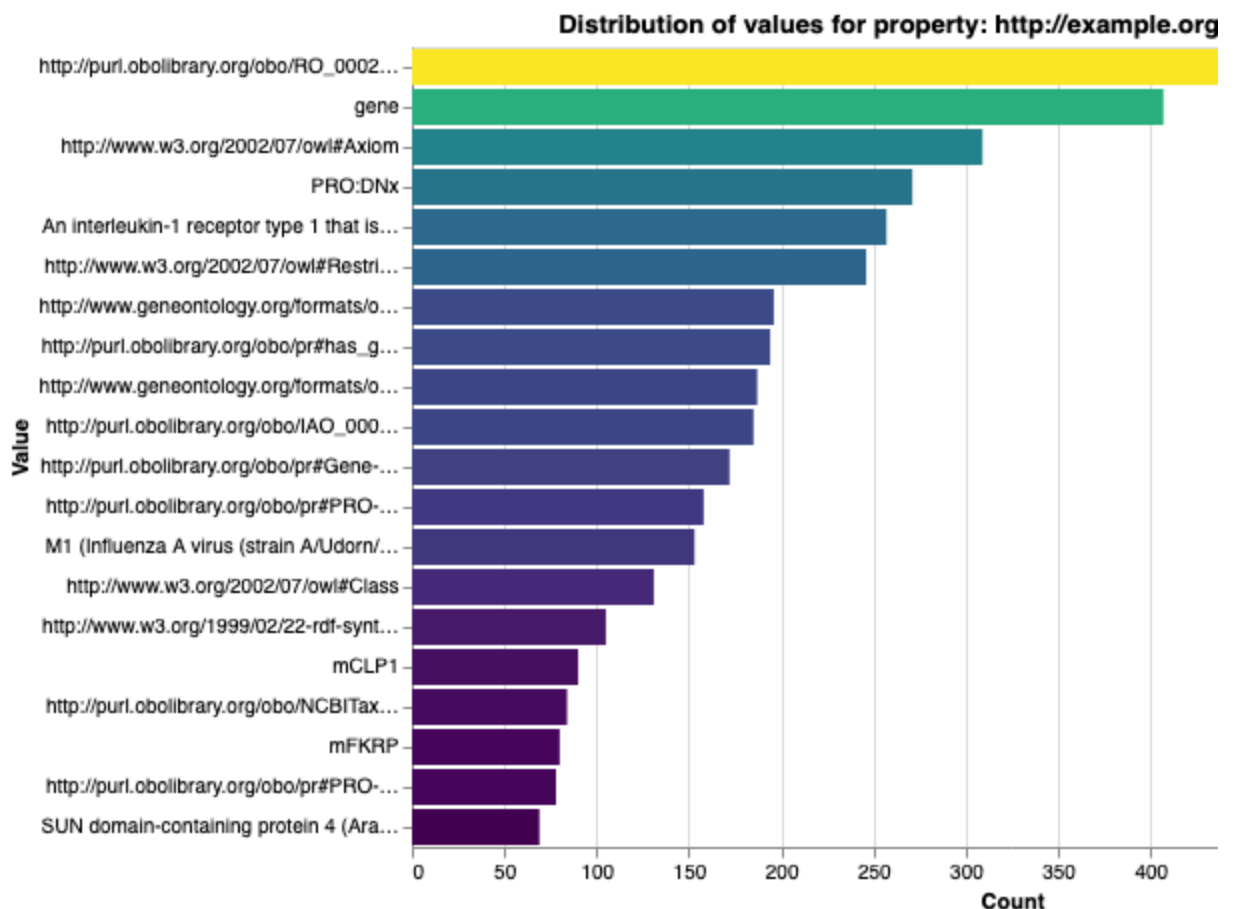
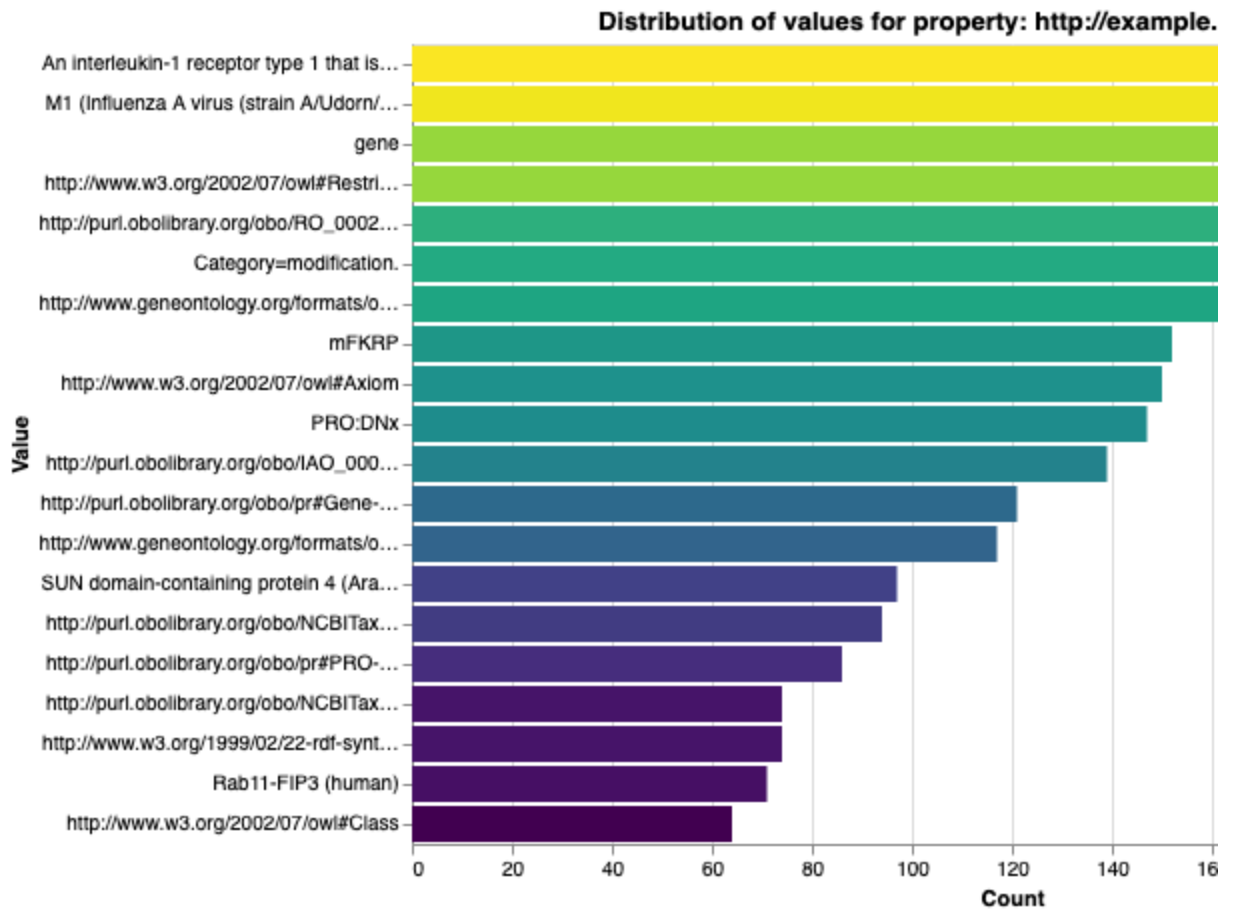
    chart.display()
```



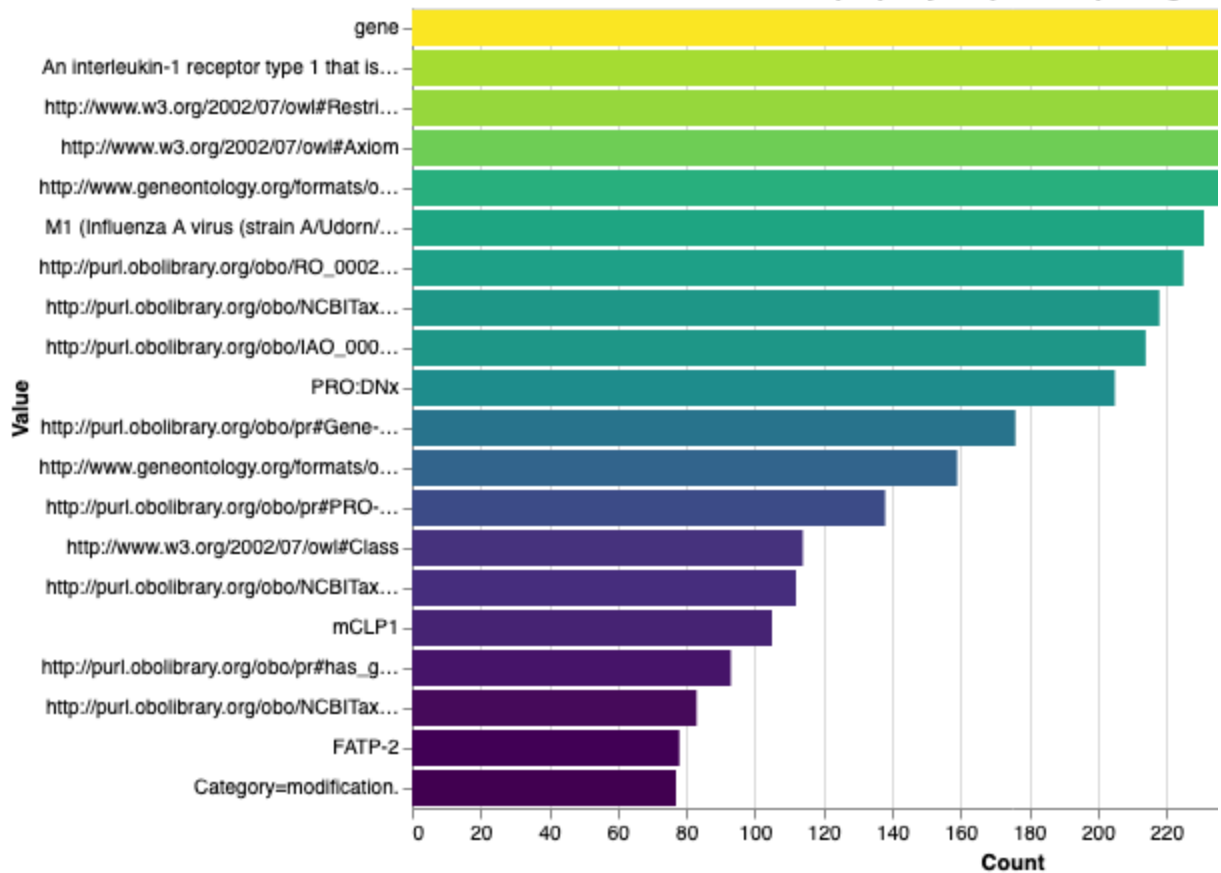




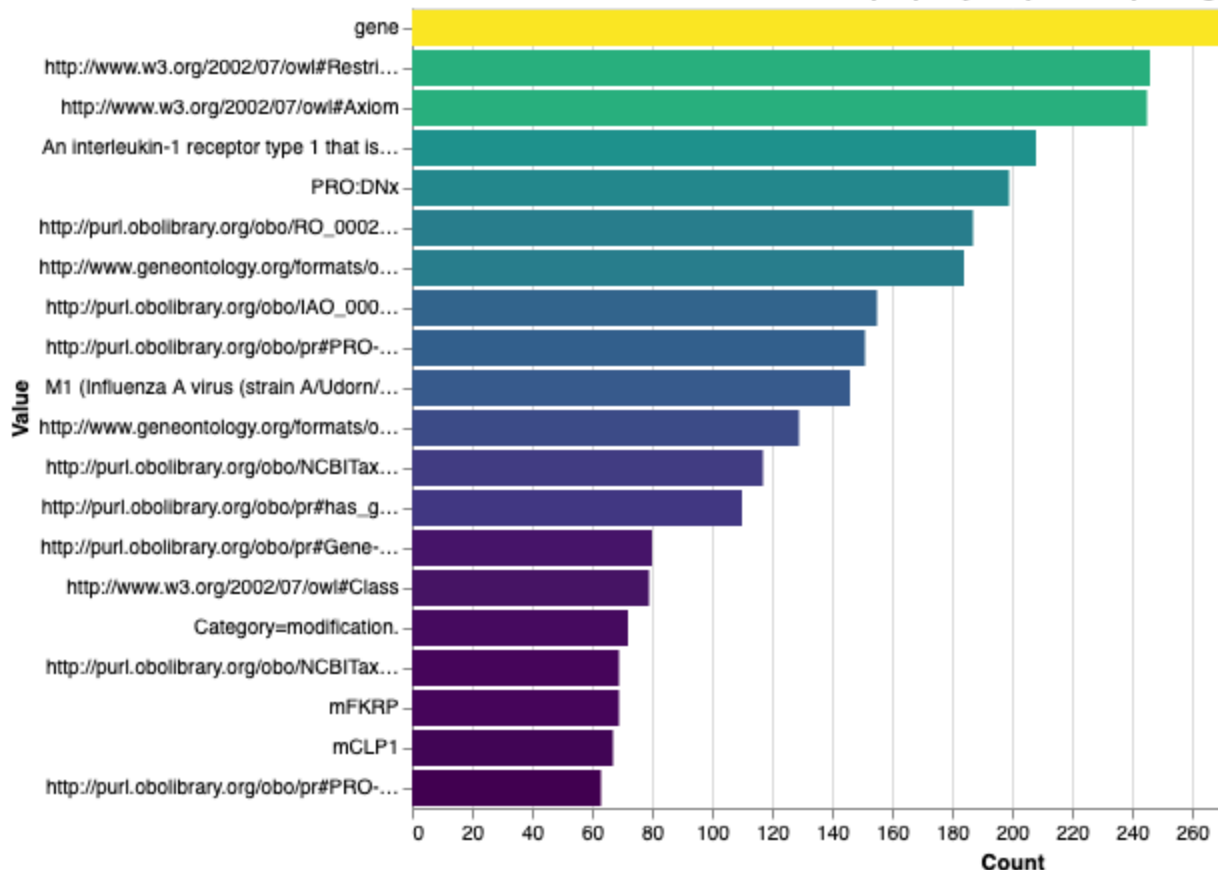


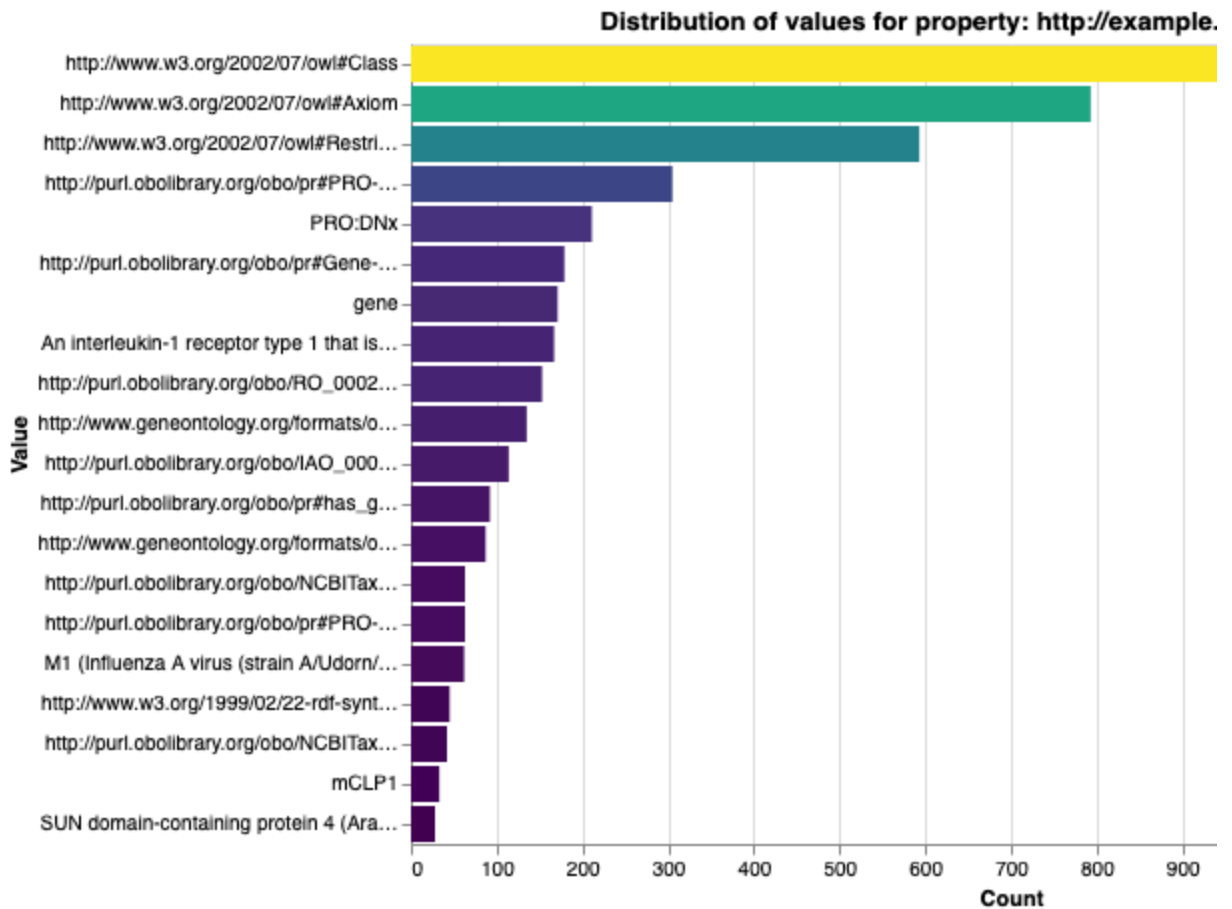


Distribution of values for property: <http://example.org/de>



Distribution of values for property: <http://example.org>





```
In [43]: from rdflib import Graph, Namespace, URIRef, BNode, Literal

# Define namespaces matching your prefixes
NS_MAP = {
    "PR": Namespace("http://purl.obolibrary.org/obo/pr#"),
    "NCBITaxon": Namespace("http://purl.obolibrary.org/obo/NCBITaxon_"),
    "RO": Namespace("http://purl.obolibrary.org/obo/RO_"),
    "oboInOwl": Namespace("http://www.geneontology.org/formats/oboInOwl#"),
    "IAO": Namespace("http://purl.obolibrary.org/obo/IAO_"),
    "ex": Namespace("http://example.org/shapes#"),
    "dcterms": Namespace("http://purl.org/dc/terms/"),
    "sh": Namespace("http://www.w3.org/ns/shacl#"),
    "rdf": Namespace("http://www.w3.org/1999/02/22-rdf-syntax-ns#"),
    "xsd": Namespace("http://www.w3.org/2001/XMLSchema#"),
    "rdfs": Namespace("http://www.w3.org/2000/01/rdf-schema#"),
    "owl": Namespace("http://www.w3.org/2002/07/owl#"),
}

DEFAULT_NS = Namespace("http://example.org/default#")

def replace_ns(uri):
    """If the URI starts with default NS, try to map to a known prefix from
    if not isinstance(uri, URIRef):
        return uri # Leave literals and BNodes unchanged

    uri_str = str(uri)
    if uri_str.startswith(str(DEFAULT_NS)):
        local_part = uri_str[len(DEFAULT_NS):]
```

```

        # Now try to detect which prefix it should have based on the local p
        # Example: PR_000011340 -> should map to PR
        # Example: NCBITaxon_9606 -> should map to NCBITaxon
        for prefix, ns in NS_MAP.items():
            if local_part.startswith(prefix + "_") or local_part.startswith(
                # Construct new URI with correct namespace + remainder
                # Remove prefix from local_part if it exists
                suffix = local_part[len(prefix):] if local_part.startswith(p
                suffix = suffix.lstrip("_") # Remove leading underscore if
                return ns[suffix]
        # If no match, fallback to original URI
        return uri
    else:
        return uri

def fix_graph_ns(g):
    new_g = Graph()
    # Bind namespaces for better serialization
    for prefix, ns in NS_MAP.items():
        new_g.bind(prefix, ns)
    new_g.bind("default", DEFAULT_NS)

    for s, p, o in g:
        s_fixed = replace_ns(s)
        p_fixed = replace_ns(p)
        o_fixed = replace_ns(o)
        new_g.add((s_fixed, p_fixed, o_fixed))
    return new_g

if __name__ == "__main__":
    g = Graph()
    g.parse("ProGo_vae_Results.ttl", format="ttl") # Replace with your TTL

    fixed_graph = fix_graph_ns(g)

    fixed_graph.serialize(destination="corrected_vae_file.ttl", format="ttl")
    print("Namespace replacement done. Saved to corrected_file.ttl")

```

Namespace replacement done. Saved to corrected_file.ttl

In [44]: `from rdflib import Graph, Namespace, URIRef, Literal, BNode`

```

# Your known namespaces
NS_MAP = {
    "PR": Namespace("http://purl.obolibrary.org/obo/pr#"),
    "NCBITaxon": Namespace("http://purl.obolibrary.org/obo/NCBITaxon_"),
    "RO": Namespace("http://purl.obolibrary.org/obo/RO_"),
    "oboInOwl": Namespace("http://www.geneontology.org/formats/oboInOwl#"),
    "IAO": Namespace("http://purl.obolibrary.org/obo/IAO_"),
    "ex": Namespace("http://example.org/shapes#"),
    "dcterms": Namespace("http://purl.org/dc/terms/"),
    "sh": Namespace("http://www.w3.org/ns/shacl#"),
    "rdf": Namespace("http://www.w3.org/1999/02/22-rdf-syntax-ns#"),
    "xsd": Namespace("http://www.w3.org/2001/XMLSchema#"),
    "rdfs": Namespace("http://www.w3.org/2000/01/rdf-schema#"),

```

```

    "owl": Namespace("http://www.w3.org/2002/07/owl#"),
    "default": Namespace("http://example.org/default#"),
}

# Map property local names (without prefix) to correct namespace prefix string
PROPERTY_TO_NS_PREFIX = {
    # from your example TTL mappings:
    "annotatedProperty": "owl",
    "annotatedSource": "owl",
    "comment": "rdfs",
    "hasBroadSynonym": "oboInOwl",
    "hasDbXref": "oboInOwl",
    "hasExactSynonym": "oboInOwl",
    "hasNarrowSynonym": "oboInOwl",
    "hasRelatedSynonym": "oboInOwl",
    "label": "rdfs",
    "onProperty": "owl",
    "someValuesFrom": "owl",
    "subClassOf": "rdfs",
    "type": "rdf",
    "created": "dcterms",
    "date": "dcterms",
    # Add more mappings here if needed
}

def fix_uri(uri):
    """Fix URIRefs that start with default: by replacing with correct NS base URI"""
    if not isinstance(uri, URIRef):
        return uri
    uri_str = str(uri)
    default_ns_str = str(NS_MAP["default"])
    if uri_str.startswith(default_ns_str):
        local_name = uri_str[len(default_ns_str):]
        # If it's a property predicate, fix namespace
        ns_prefix = PROPERTY_TO_NS_PREFIX.get(local_name)
        if ns_prefix and ns_prefix in NS_MAP:
            return NS_MAP[ns_prefix][local_name]
        else:
            # Could be a resource, keep default or handle differently
            # Optionally fallback to default namespace
            return uri
    else:
        return uri

def fix_predicate(pred):
    """Fix predicate URIs with default prefix by mapping local name to correct NS"""
    if not isinstance(pred, URIRef):
        return pred
    pred_str = str(pred)
    default_ns_str = str(NS_MAP["default"])
    if pred_str.startswith(default_ns_str):
        local_name = pred_str[len(default_ns_str):]
        ns_prefix = PROPERTY_TO_NS_PREFIX.get(local_name)
        if ns_prefix and ns_prefix in NS_MAP:
            return NS_MAP[ns_prefix][local_name]
        else:

```

```

        # fallback, keep original
        return pred
    else:
        return pred

def fix_object(obj):
    """Fix object URIs similarly if they start with default prefix."""
    if isinstance(obj, URIRef):
        return fix_uri(obj)
    return obj # literals or blank nodes unchanged

def fix_graph(g):
    new_g = Graph()
    # Bind namespaces for better output
    for prefix, ns in NS_MAP.items():
        new_g.bind(prefix, ns)

    for s, p, o in g:
        p_fixed = fix_predicate(p)
        o_fixed = fix_object(o)
        new_g.add((s, p_fixed, o_fixed))
    return new_g

if __name__ == "__main__":
    g = Graph()
    g.parse("corrected_vae_file.ttl", format="ttl") # your TTL input file

    fixed_g = fix_graph(g)
    fixed_g.serialize("corrected_file.ttl", format="ttl")
    print("Done! Output saved to corrected_file.ttl")

```

Done! Output saved to corrected_file.ttl

```

In [87]: from rdflib import Graph, Namespace, URIRef, Literal
        from collections import defaultdict, Counter
        import pandas as pd

        # Load your RDF data file
        g = Graph()
        g.parse("Pro_Go_gan_file.ttl", format="turtle") # <-- Replace with your RDF

        # Define namespaces
        RDF = Namespace("http://www.w3.org/1999/02/22-rdf-syntax-ns#")
        oboInOwl = Namespace("http://www.geneontology.org/formats/oboInOwl#")
        rdfs = Namespace("http://www.w3.org/2000/01/rdf-schema#")
        owl = Namespace("http://www.w3.org/2002/07/owl#")
        PR = Namespace("http://purl.obolibrary.org/obo/pr#")

        # Bind namespaces (optional for nicer output)
        g.bind("rdf", RDF)
        g.bind("oboInOwl", oboInOwl)
        g.bind("rdfs", rdfs)
        g.bind("owl", owl)
        g.bind("pr", PR)

        # Old and new base URI strings for protein ID replacement

```

```

OLD_BASE = "http://purl.obolibrary.org/obo/pr#"
NEW_BASE = "https://proconsortium.org/cgi-bin/entry_pro?id="

# Properties to extract (using URIRefs)
properties_to_extract = [
    owl.annotatedProperty,
    owl.annotatedSource,
    rdfs.comment,
    oboInOwl.hasBroadSynonym,
    oboInOwl.hasDbXref,
    oboInOwl.hasExactSynonym,
    oboInOwl.hasNarrowSynonym,
    oboInOwl.hasRelatedSynonym,
    rdfs.label,
    owl.onProperty,
    owl.someValuesFrom,
    rdfs.subClassOf,
    RDF.type
]

# Dictionary to store values per protein_id and property
protein_data = defaultdict(lambda: defaultdict(list))
protein_ids_found = set()

# Iterate over all triples and extract relevant data
for subj, pred, obj in g:
    if pred not in properties_to_extract:
        continue

    subj_str = str(subj)

    # Process only subjects starting with the old protein URI prefix
    if subj_str.startswith(OLD_BASE):
        # Extract the part after the prefix, e.g. "000000453_1"
        sample_part = subj_str[len(OLD_BASE):]

        # Extract base protein ID before underscore (e.g., "000000453")
        base_id = sample_part.split('_')[0]

        protein_ids_found.add(base_id)

        # Create new protein URI with the new base + base_id
        protein_uri = NEW_BASE + base_id

        # Convert object to string (URIRefs or Literals)
        if isinstance(obj, Literal):
            value_str = str(obj)
        elif isinstance(obj, URIRef):
            value_str = str(obj)
        else:
            value_str = str(obj)

        # Append the value grouped by protein and property
        protein_data[protein_uri][pred].append(value_str)

# Print all unique base protein IDs found

```



```

print(f"Found base protein IDs ({len(protein_ids_found)}): {sorted(protein_ids_found)}")

# Prepare list to hold statistics
stats = []

# Calculate stats per protein and property
for protein, props in protein_data.items():
    for prop, values in props.items():
        # Flatten comma-separated values (some values may be multiple, e.g. "1,2,3")
        flat_values = []
        for v in values:
            if ',' in v:
                flat_values.extend([x.strip() for x in v.split(',')])
            else:
                flat_values.append(v)

        counts = Counter(flat_values)
        total = sum(counts.values())
        unique_count = len(counts)
        most_common_val, most_common_count = counts.most_common(1)[0] if unique_count > 0 else None

        stats.append({
            "protein_id": protein,
            "property": prop.n3(g.namespace_manager), # Pretty QName like c
            "unique_values_count": unique_count,
            "most_common_value": most_common_val,
            "most_common_count": most_common_count,
            "value_counts": dict(counts),
            "total_values": total
        })

# Create a DataFrame for easy viewing or export
df_stats = pd.DataFrame(stats)

# Display the stats
print(df_stats)

# Optional: Save to CSV if you want
# df_stats.to_csv("protein_property_stats.csv", index=False)

```

Found base protein IDs (1391): ['000000453', '000000843', '000000992', '000001167', '000001705', '000001844', '000001986', '000001987', '000002407', '000003174', '000003200', '000003249', '000003450', '000003452', '000003766', '000003923', '000004110', '000004467', '000004690', '000004882', '000004888', '000004896', '000005333', '000005353', '000005477', '000005651', '000005826', '000005842', '000006122', '000006146', '000006180', '000006220', '000006417', '000006699', '000006714', '000006781', '000006803', '000006847', '000007104', '000007623', '000007744', '000007810', '000007920', '000008143', '000008299', '000008934', '000008984', '000009025', '000009033', '000009329', '000009884', '000010012', '000010131', '000010428', '000010820', '000010940', '000011013', '000011249', '000011370', '000011492', '000011802', '000012021', '000012193', '000012212', '000012217', '000012225', '000012788', '000012973', '000013107', '000013136', '000013345', '000013476', '000013701', '000014253', '000014310', '000014319', '000014364', '000014928', '000015046', '000015210', '000015360', '000015502', '000015640', '000015721', '000015782', '000016112', '000016173', '000016260', '000016278', '000016674', '000016681', '000016866', '000016997', '000017307', '000017810', '000017943', '000018252', '000018863', '000018941', '000019006', '000019011', '000019103', '000019236', '000019490', '000019608', '000020241', '000020412', '000020458', '000020555', '000020643', '000020835', '000021195', '000021214', '000021465', '000021520', '000021540', '000021604', '000021685', '000021920', '000022255', '000022859', '000023919', '000024140', '000024176', '000024244', '000024397', '000024472', '000024598', '000024985', '000025278', '000025403', '000025786', '000025926', '000026089', '000026552', '000026766', '000027133', '000027200', '000027493', '000027664', '000027799', '000027959', '000028492', '000028509', '000028654', '000029166', '000029223', '000029268', '000029316', '000029436', '000029522', '000029717', '000029766', '000029802', '000029949', '000030057', '000030220', '000030259', '000030304', '000030605', '000031183', '000031419', '000031613', '000031881', '000032249', '000032268', '000032328', '000032331', '000032512', '000032742', '000033146', '000033731', '000033757', '000034221', '000035931', '000036075', '000036175', '000036224', '000036497', '000036869', '000038174', '000038874', '000039152', '000039337', '000039411', '000039863', '000040294', '000040396', '000040558', '000040708', '000040737', '000040788', '000040930', '000040956', '000042085', '000042667', '000042780', '000042895', '000043129', '000043204', '000043877', '000043949', '000044141', '000044509', '000044846', '000045034', '000045217', '000045305', '000045397', '000045469', '000045681', '000045856', '000045885', '000045890', '000046348', '000046393', '000046685', '000046690', '000046694', '000047169', '000047240', '000047325', '000047496', '000047512', '000047670', '000047868', '000048805', '000048934', '000048943', '000049145', '000049292', '000049382', '000049524', '000049633', '000049936', '000050569', '000050684', '000050779', '000051299', '000051370', '000051378', '000051951', '000051994', '000052387', '000052409', '000052627', '000052707', '000052767', '000053198', '000053304', '000053331', '000053504', '000053581', '000053875', '000053959', '000054043', '000054320', '000054509', '000055120', '000055317', '000055349', '000055584', '000055639', '000055665', '000055885', '000055941', '000055973', '000056082', '000056312', '000056407', '000056589', '000057003', '000057037', '000057040', '000057447', '000057542', '000057744', '000057769', '000057887', '000057994', '000058021', '000058226', '000058985', '000058995', '000059155', '000059302', '000059597', '000059807', '000060118', '000060294', '000060478', '000060588', '000061212', '000061662', '000061808', '000062579', '000062872', '000062888', '000062994', '000063115', '000063441', '000063527', '000063761', '000064487', '000064840', '000064844', '000064895', '000065042', '000065223', '000065318', '000065321', '000065322', '000065392', '000065733', '000065781', '000065854', '000066724', '000066954', '000067167', '000067478', '000067718', '000067824', '0000

67889', '000068013', '000068856', '000068877', '000069022', '000069518', '000069690', '000069960', '000070471', '000070682', '000070862', '000070948', '000071280', '000071439', '000071665', '000071908', '000072034', '000072342', '000072491', '000072567', '000072832', '000073030', '000073159', '000073214', '000073504', '000074469', '000074582', '000075564', '000075860', '000076166', '000076716', '000076842', '000077093', '000077465', '000077559', '000078013', '000078309', '000078337', '000078501', '000078680', '000078836', '000079025', '000079849', '000079909', '000080154', '000080463', '000080761', '000081038', '000081191', '000081339', '000081959', '000082460', '000082527', '000083196', '000083235', '000083406', '000083571', '000083689', '000083693', '000083931', '000084140', '000084386', '000084534', '000084598', '000084650', '000084725', '000085070', '000085667', 'A0A0B4J273', 'A0A0G2JXN2', 'A0A0H3G1J6', 'A0A163UT06-2', 'A0A1B0GUA6', 'A0A494C0N9', 'A0AV02', 'A0FKI7', 'A0KEM9', 'A0M8U1-1', 'A0MZ66-5', 'A1A6M1-1', 'A2AFR3-1', 'A2ARI4-2', 'A2ATU0', 'A2VEC9', 'A3CK62', 'A3KPA3-1', 'A3KQ9', 'A4K144', 'A5HY34', 'A6NHM9', 'A7E2F4-3', 'A7XDQ9', 'A8K8P3', 'A8K8P3-3', 'A8MQ11', 'B3DHS1', 'B4F7E7', 'B6SEH8', 'B8A4W9-2', 'B9EJV3', 'C0LGH3-4', 'C6KI89', 'C6Y4C1', 'D2WL32-1', 'D3ZNJ5', 'D4ABW7', 'D4AEI0', 'E1BQH0', 'E7EZQ7', 'E7F590', 'E7FGH8', 'E9Q236-1', 'F1P0E2', 'F1Q575', 'F4HRV8', 'F4I111', 'F4I8I0', 'F4IHY7', 'F4ITM1', 'F4J7T2', 'F4JFN3', 'F4JRJ6', 'F4JSZ5', 'F4JTN2-1', 'F4K2U8', 'G5EFF5-4', 'H9L3K3', '000194-1', '008650', '013898', '014118', '014164', '014342', '015021', '015021-4', '015034', '015393', '022622', '031465', '035134', '035350', '035600', '042957', '043497-3', '043602-1', '043731-1', '043861-1', '043897-1', '043913', '044509', '044743', '054775-1', '054874-1', '054874-2', '054937', '055106', '055225-2', '060268', '060359', '060359-1', '060645', '064782', '070257', '073592', '074254', '074455', '074525', '074862', '074968', '075072-2', '075154', '075325', '075431-1', '075838', '076036', '076036-5', '076460', '080340', '080758', '082233', '082277', '088466', '088763', '088828', '088943', '088943-9', '093327-1', '094504', '094560', '094886', '095057-1', '095076', '095168-1', '095249', '095273-4', '095436-1', '095684-3', 'P00452', 'P00547', 'P01143', 'P01244-1', 'P01733', 'P02647', 'P02683', 'P04692-3', 'P05164', 'P05182', 'P05556-5', 'P05745', 'P06986', 'P07464', 'P07658', 'P08092', 'P08201', 'P08390', 'P08487', 'P08493', 'P08582-1', 'P08910', 'P09450-1', 'P0A7P2', 'P0A870', 'P0A921', 'P0A9J6', 'P0AAH0', 'P0AAI9', 'P0AD86', 'P0AF12', 'P0AF34', 'P0AGK1', 'P0C0W1', 'P0C2N6', 'P0C5U1', 'P0C6H2', 'P0CC09', 'P0CG43', 'P0CJ88', 'P0CX46', 'P0DI19', 'P0DMQ9', 'P0DOA7', 'P0DOF4', 'P10073', 'P10151', 'P10634', 'P10683-1', 'P10862', 'P12003', 'P12109', 'P12494', 'P12804', 'P13107', 'P13866', 'P14327', 'P14565', 'P15692-9', 'P16220-3', 'P16423', 'P17021-2', 'P17568', 'P17678', 'P17743', 'P18577', 'P18577-10', 'P19332-3', 'P19627-1', 'P20138-1', 'P20228', 'P20371', 'P20482-2', 'P20720', 'P20810', 'P20810-3', 'P20871', 'P20963-3', 'P21439-2', 'P21643-1', 'P22137', 'P22467', 'P23512', 'P23524', 'P23711', 'P24171', 'P24604-6', 'P25332', 'P25378', 'P25425-7', 'P25918', 'P26569-1', 'P28076', 'P28166-2', 'P28296', 'P28519', 'P28648-1', 'P28769', 'P29117-1', 'P30040', 'P30101', 'P30510', 'P30913', 'P30940', 'P31150', 'P31362', 'P32314-2', 'P32634', 'P32662', 'P32769', 'P32909', 'P33766', 'P34110', 'P34221', 'P34244', 'P34333-2', 'P34370', 'P35169', 'P35294-1', 'P35441-1', 'P35658', 'P36137', 'P37231-2', 'P37634', 'P37747', 'P38616', 'P40314', 'P40337-3', 'P40356', 'P40962', 'P41036', 'P41161', 'P41247-2', 'P41778', 'P42685-2', 'P42701', 'P43255', 'P43601', 'P43699', 'P45549', 'P46139', 'P46600', 'P47088', 'P48349', 'P49866-2', 'P50273', 'P50463', 'P51790-2', 'P52179', 'P52306-5', 'P52744', 'P52747-2', 'P53193', 'P53283', 'P53287', 'P53349', 'P53494', 'P54099', 'P54310', 'P54357', 'P54362', 'P54730', 'P54904', 'P55017-3', 'P55083', 'P55168-1', 'P55314-1', 'P55808', 'P56402', 'P56402-1', 'P56574', 'P56574-1', 'P56628', 'P56696-2', 'P56840', 'P56842', 'P57075', 'P58048', 'P58340-2', 'P58512', 'P60390', 'P61086-2', 'P61201', 'P61201-1', 'P61363', 'P61765', 'P61866', 'P61979-2',

'P62138', 'P62482-1', 'P62501', 'P62744', 'P62906', 'P62919-1', 'P63213', 'P65556', 'P69783', 'P69922', 'P70310-1', 'P70389', 'P70451-5', 'P75024', 'P75855', 'P75906', 'P76397', 'P76549', 'P77339', 'P80365', 'P80967', 'P81277', 'P82729', 'P84243', 'P84247', 'P85411-2', 'P87052', 'P87058', 'P91622', 'P92550', 'P93002', 'P97310-1', 'P97450', 'P97471', 'P97686-3', 'P97927', 'P97929', 'P9WJF5', 'Q01813-1', 'Q01826', 'Q02447-4', 'Q02721', 'Q02734', 'Q02883', 'Q03555', 'Q03823', 'Q03942', 'Q04836', 'Q04836-1', 'Q04861-1', 'Q04941', 'Q05319', 'Q05860-1', 'Q06011', 'Q06524', 'Q06632', 'Q07157-1', 'Q07187', 'Q07343', 'Q07343-3', 'Q07409-1', 'Q08004', 'Q08202', 'Q08281', 'Q08761', 'Q09013-9', 'Q09173', 'Q09399', 'Q09466', 'Q09712', 'Q09874', 'Q0V869-1', 'Q0VBD2', 'Q0VBF8', 'Q0WLB5', 'Q10251', 'Q10315', 'Q10454-2', 'Q10474', 'Q11203-18', 'Q12234', 'Q12349', 'Q12999', 'Q13077-1', 'Q13439-1', 'Q13490', 'Q13639-2', 'Q14188-8', 'Q14204', 'Q14677', 'Q14680', 'Q14680-2', 'Q14689', 'Q14C87-2', 'Q15329-2', 'Q15485-1', 'Q16288', 'Q16720-5', 'Q16827-2', 'Q16881', 'Q16881-2', 'Q16890-5', 'Q17902-2', 'Q19127', 'Q19338-2', 'Q19426', 'Q19584', 'Q19746', 'Q1ERP8-1', 'Q1H5D2', 'Q1JXP4', 'Q1L981-2', 'Q1T7C0', 'Q22701', 'Q23490', 'Q24306', 'Q27873', 'Q2EES1', 'Q2FVJ6', 'Q2FWD0', 'Q2FYF1', 'Q2FZG8', 'Q2G1T9', 'Q2G272', 'Q2G2C3', 'Q2G2U8', 'Q2M1K6', 'Q2TAA8-2', 'Q32B37', 'Q32ZH2', 'Q38836', 'Q39043', 'Q39231', 'Q3E7X9', 'Q3SWU0', 'Q3SY56', 'Q3TQ03', 'Q3TVW5', 'Q3UA16-1', 'Q3UFY7', 'Q3UP75', 'Q3V0F0', 'Q401N2-1', 'Q495B1', 'Q499M7', 'Q4V8F5', 'Q4VNC1-1', 'Q53FA7-2', 'Q53FD0', 'Q54CR2', 'Q54CY3', 'Q54D00', 'Q54DK4', 'Q54DV3', 'Q54GG1', 'Q54GV0', 'Q54IH6', 'Q54J55', 'Q54KN4', 'Q54MW0', 'Q54ND3', 'Q54SW1', 'Q54TS4', 'Q54VV8', 'Q54WW2', 'Q54X95', 'Q54XC4', 'Q55AN8', 'Q55B99', 'Q55E34', 'Q55FT4', 'Q562E7-4', 'Q576K0', 'Q59LV5', 'Q59MW2', 'Q59ST1', 'Q5A861', 'Q5A8X9', 'Q5AEM5', 'Q5BJH7', 'Q5BK09', 'Q5BK09-1', 'Q5EB66', 'Q5EBL4-3', 'Q5F4A3', 'Q5GH73-1', 'Q5H8A4', 'Q5H8A4-4', 'Q5H9K5', 'Q5H9K5-1', 'Q5HZ60-2', 'Q5I0E6-1', 'Q5JR59', 'Q5JUR7-1', 'Q5PQN6', 'Q5PR00-1', 'Q5PRC1-1', 'Q5T0F9-4', 'Q5T5B0', 'Q5U2S5', 'Q5XF82', 'Q5XIA4-1', 'Q5ZHU0', 'Q5ZIW2', 'Q5ZLS8', 'Q60936', 'Q61169', 'Q61578', 'Q62433', 'Q62433-1', 'Q62448', 'Q62504', 'Q62767', 'Q62927', 'Q63447', 'Q63449-2', 'Q63515', 'Q63767', 'Q640N2', 'Q64481', 'Q64649', 'Q64727-1', 'Q66GR0', 'Q66I61', 'Q68DK2', 'Q68DU8', 'Q68FT3-1', 'Q69CM7', 'Q6AYT2-1', 'Q6BEB4-1', 'Q6DBA5', 'Q6DBU8', 'Q6DD88-1', 'Q6DHF7', 'Q6DKJ4', 'Q6DPU1', 'Q6DR20', 'Q6DRN3', 'Q6GQ09', 'Q6GTS8-1', 'Q6HE08', 'Q6IA86', 'Q6IML7', 'Q6IN36-1', 'Q6IQX0-2', 'Q6JHU7', 'Q6LA42', 'Q6NXX2-1', 'Q6NXP6-1', 'Q6P1L6', 'Q6P1N9-1', 'Q6P773', 'Q6P9Q6', 'Q6PDI6', 'Q6PHF0', 'Q6Q1P4', 'Q6R8J2', 'Q6SJP8', 'Q6TYB5', 'Q6UX73', 'Q6UXI7', 'Q6UXV0', 'Q6UXY8-1', 'Q6V7V2', 'Q6VMQ6', 'Q6VYA3', 'Q6W5C0', 'Q6ZNA4-4', 'Q6ZPQ6-1', 'Q6ZPV2-1', 'Q6ZS11', 'Q6ZUT6', 'Q6ZVX7', 'Q700D5', 'Q70EL3', 'Q71UK2', 'Q75JR2', 'Q792H5-1', 'Q7L3V2', 'Q7L5Y6', 'Q7T322', 'Q7TPQ3-5', 'Q7TQC5-3', 'Q7TQP0', 'Q7TQP4', 'Q7Y208', 'Q7YTU4', 'Q7Z353-1', 'Q7Z401', 'Q7Z407', 'Q7Z407-3', 'Q7Z4H4', 'Q7Z6G3', 'Q7Z6W7', 'Q7ZW47', 'Q800Q5', 'Q803M3', 'Q803N9', 'Q80T03', 'Q80TM6', 'Q80TM6-3', 'Q80UP3', 'Q80VC9-8', 'Q80XP8-1', 'Q84M91', 'Q86BY9', 'Q86KI1', 'Q86U86-8', 'Q86VP1-2', 'Q86XL3', 'Q86XL3-3', 'Q86XR8', 'Q86YW5-2', 'Q89WE8', 'Q8BFZ6', 'Q8BHE5-1', 'Q8BII1', 'Q8BK06-3', 'Q8BMG7', 'Q8BN57', 'Q8BN59', 'Q8BP67', 'Q8BSA9', 'Q8BT18', 'Q8BTM9', 'Q8BWU5-1', 'Q8BXS7', 'Q8BZI0', 'Q8BZI0-2', 'Q8BZK4', 'Q8BZP8', 'Q8C0R7-2', 'Q8C0Y0-2', 'Q8C569', 'Q8C650', 'Q8C7K6', 'Q8C7U7', 'Q8CAQ8-2', 'Q8CFC4', 'Q8CG64', 'Q8CHJ1-2', 'Q8CJ19-3', 'Q8CWS0', 'Q8H129-1', 'Q8H1D6', 'Q8I4B0-3', 'Q8I8U7-1', 'Q8IQ70-1', 'Q8IS44-5', 'Q8IU81', 'Q8IU81-1', 'Q8IUF8', 'Q8IUQ4-2', 'Q8IUY3', 'Q8IVV2-1', 'Q8IWW8-2', 'Q8IXF0-4', 'Q8IY84-1', 'Q8IYU2-2', 'Q8IZF5-1', 'Q8K013', 'Q8K203', 'Q8K2P6', 'Q8K4Q6-1', 'Q8L5A7', 'Q8LEA2', 'Q8MXJ9', 'Q8N3F0', 'Q8N3F0-3', 'Q8N6K7', 'Q8N967-1', 'Q8N9H8-2', 'Q8N9M1', 'Q8NB16-2', 'Q8NDH3', 'Q8NEZ4-1', 'Q8NFV4-2', 'Q8NH94', 'Q8NI17-7', 'Q8QG60', 'Q8R087', 'Q8R104-1', 'Q8R1N4', 'Q8R2K1-3', 'Q8R2N2', 'Q8R4R9', 'Q8R5M8-1', 'Q8RIE4', 'Q8SSQ0', 'Q8TAC9', 'Q8TAC9-1', 'Q8TAI7-2', 'Q8TAS1', 'Q8TAV3', 'Q8TB24', 'Q8TB36', 'Q8TB36-1', 'Q8TE96-3', 'Q8VBV7', 'Q8VBW

1', 'Q8VCL5', 'Q8VHI7-2', 'Q8VHJ7-1', 'Q8VWF1', 'Q8VXX4', 'Q8VY15', 'Q8VY74-2', 'Q8W0Y8-1', 'Q8W2F2-2', 'Q8W4P1-1', 'Q8WVK7', 'Q8WVM7', 'Q8WVN8', 'Q8WW14', 'Q8WW14-1', 'Q8WWK9-5', 'Q8WXJ9-1', 'Q8WXR4-3', 'Q8YGC2', 'Q90257', 'Q90705', 'Q91048', 'Q91VK4', 'Q91W34-1', 'Q91W52', 'Q91WN2', 'Q91XU3-1', 'Q91YI1', 'Q91YL3', 'Q91ZA8', 'Q92008', 'Q921W4', 'Q922K7', 'Q92339', 'Q923B6', 'Q924X6-5', 'Q92567-2', 'Q92570', 'Q92581-2', 'Q92597-2', 'Q92973-3', 'Q93038-8', 'Q93Z20', 'Q93ZV7', 'Q94527', 'Q949U7', 'Q94FL6', 'Q95RW8', 'Q95RW8-2', 'Q968Z6', 'Q969I3', 'Q969X0', 'Q969X1', 'Q96AG3', 'Q96BI1', 'Q96BM9-1', 'Q96BR1', 'Q96DX7', 'Q96EF0', 'Q96FC9', 'Q96GM1', 'Q96GZ6-5', 'Q96H35-1', 'Q96IX5-1', 'Q96JN2-2', 'Q96L11-1', 'Q96L34-1', 'Q96LZ7', 'Q96MP8', 'Q96N21-1', 'Q96NW4', 'Q96NX5', 'Q96NZ8', 'Q96PZ7', 'Q96PZ7-4', 'Q96QE3', 'Q96QF0', 'Q96RP3', 'Q96RR1-2', 'Q96T76-7', 'Q99369', 'Q99471-1', 'Q99JR8', 'Q99JY3', 'Q99JY3-1', 'Q99K10-1', 'Q99K74', 'Q99LI9', 'Q99LT0', 'Q99MK8', 'Q99NE9', 'Q99NE9-2', 'Q99P72', 'Q9ASY9-2', 'Q9ATB4-2', 'Q9BKQ5', 'Q9BQA9-1', 'Q9BQS8-1', 'Q9BRT8-4', 'Q9BTE1', 'Q9BUY5', 'Q9BV97', 'Q9BX51', 'Q9BXE9', 'Q9BXW9', 'Q9BZI7', 'Q9C0W3', 'Q9C5D3-1', 'Q9C5U3', 'Q9C882-2', 'Q9C9Z9', 'Q9CQ54-1', 'Q9CY25', 'Q9CZX8', 'Q9D3W4', 'Q9D6X6', 'Q9D8C6-1', 'Q9DAN1', 'Q9DAS4-2', 'Q9DB42-1', 'Q9DBI0', 'Q9DCQ7', 'Q9DD02-2', 'Q9EPL8', 'Q9EQJ9', 'Q9EQV6', 'Q9FMY7', 'Q9FNC1', 'Q9FQ09', 'Q9FV71-2', 'Q9FX77', 'Q9FXA3', 'Q9FYE5', 'Q9FYL2', 'Q9GZT3-2', 'Q9H2D1', 'Q9H2S9-1', 'Q9H305-2', 'Q9H5L6', 'Q9H7T0', 'Q9HB75-4', 'Q9HBB1', 'Q9HBBK9', 'Q9HBBX9-4', 'Q9HDW1', 'Q9HGG2', 'Q9HUD0', 'Q9I8F4', 'Q9JI60', 'Q9JI67', 'Q9JJ69-1', 'Q9JKT5', 'Q9JLV6', 'Q9LDI3', 'Q9LET3', 'Q9LF50', 'Q9LH43', 'Q9LN69', 'Q9LUR3-1', 'Q9LUS2', 'Q9LVP0', 'Q9LY08-2', 'Q9LYT7', 'Q9M063', 'Q9M1D0', 'Q9M1D1', 'Q9M1E2', 'Q9M2J2', 'Q9M2N5', 'Q9M2Z4', 'Q9M338', 'Q9M9L6', 'Q9NFU0', 'Q9NPF2', 'Q9NQ11-3', 'Q9NRA1', 'Q9NRD9-2', 'Q9NRR4-1', 'Q9NRX2', 'Q9NS87', 'Q9NS87-4', 'Q9NSI2-2', 'Q9NWS1-3', 'Q9NWW4', 'Q9NYT6', 'Q9NZ81-1', 'Q9NZH8', 'Q9NZQ0', 'Q9NZR1-2', 'Q9P016-1', 'Q9P2R3', 'Q9P6K3', 'Q9PTR5', 'Q9QXC1', 'Q9QXE4-2', 'Q9QXK7', 'Q9QXT6-1', 'Q9QY36-1', 'Q9QYE3-10', 'Q9QYF9', 'Q9QYZ8-1', 'Q9QZ47-5', 'Q9QZC7', 'Q9R098', 'Q9R190-1', 'Q9R1T4-1', 'Q9R244-4', 'Q9S7G6', 'Q9S7U5', 'Q9S7Z2', 'Q9S9N4', 'Q9SA16', 'Q9SA34', 'Q9SAH2', 'Q9SAU2', 'Q9SD40-1', 'Q9SI06', 'Q9SIA1', 'Q9SII0', 'Q9SK50', 'Q9SQQ9', 'Q9SQQ9-1', 'Q9SQT9', 'Q9SR70', 'Q9SS44', 'Q9STE8', 'Q9SU92', 'Q9SVU4', 'Q9SZ13', 'Q9SZT9-2', 'Q9T0I6', 'Q9U1W2-2', 'Q9UBC3-4', 'Q9UHH9-1', 'Q9UI30', 'Q9UI38', 'Q9UK32', 'Q9UKE5-5', 'Q9ULC0-2', 'Q9UNK9', 'Q9UQ88-10', 'Q9USW5', 'Q9V7U0', 'Q9V7U0-1', 'Q9V9Y4', 'Q9VDE4', 'Q9VIQ9', 'Q9VJQ5', 'Q9VNA4', 'Q9VUJ0', 'Q9VVK8', 'Q9VVT9', 'Q9W3E1', 'Q9W3E2', 'Q9W5Y0', 'Q9WTN5-1', 'Q9WV92-2', 'Q9WVG7', 'Q9WVQ0', 'Q9XJ36', 'Q9XZI6-2', 'Q9Y215-3', 'Q9Y275-1', 'Q9Y2R9', 'Q9Y4P8-1', 'Q9Y597-1', 'Q9Y5L0', 'Q9Y5L0-2', 'Q9Y5P3', 'Q9Y5Q0', 'Q9Y6B7', 'Q9Z0E2', 'Q9Z173-2', 'Q9Z1L3', 'Q9Z1N4', 'Q9Z351-6', 'Q9ZNX9', 'Q9ZPV7-1', 'Q9ZSR7-1', 'Q9ZVF5', 'Q9ZVQ3', 'Q9ZVR0', 'Q9ZW19', 'W6RTA4-3']

	protein_id \
0	https://proconsortium.org/cgi-bin/entry_pro?id...
1	https://proconsortium.org/cgi-bin/entry_pro?id...
2	https://proconsortium.org/cgi-bin/entry_pro?id...
3	https://proconsortium.org/cgi-bin/entry_pro?id...
4	https://proconsortium.org/cgi-bin/entry_pro?id...
...	...
18078	https://proconsortium.org/cgi-bin/entry_pro?id...
18079	https://proconsortium.org/cgi-bin/entry_pro?id...
18080	https://proconsortium.org/cgi-bin/entry_pro?id...
18081	https://proconsortium.org/cgi-bin/entry_pro?id...
18082	https://proconsortium.org/cgi-bin/entry_pro?id...

	property	unique_values_count \
0	oboInOwl:hasNarrowSynonym	8

1	rdfs:comment	2
2	rdfs:subClassOf	2
3	rdf:type	4
4	rdfs:label	3
...	...	...
18078	owl:onProperty	4
18079	rdfs:comment	4
18080	oboInOwl:hasDbXref	1
18081	oboInOwl:hasRelatedSynonym	3
18082	owl:annotatedProperty	2

	most_common_value	most_common_count	\
0	http://purl.obolibrary.org/obo/PR_000013537	2	
1	Rab11-FIP3 (human)	5	
2	UniProtKB:P51587	5	
3	http://purl.obolibrary.org/obo/pr#000047868	5	
4	http://purl.obolibrary.org/obo/NCBITaxon_10090	3	
...	...	...	...
18078	gene	4	
18079	n7f59374c633d4112829413c038342ae5b3402	3	
18080	PRO:DNx	5	
18081	http://www.w3.org/1999/02/22-rdf-syntax-ns#nil	5	
18082	Glce	5	

	value_counts	total_values
0	{'PRO:CNA': 1, 'Araport:AT1G63470': 1, 'http://...}	10
1	{'Rab11-FIP3 (human)': 5, 'http://www.w3.org/2...}	6
2	{'UniProtKB:P51587': 5, '1-2400': 5}	10
3	{'hRUFY1/Phos:1': 1, 'http://purl.obolibrary.o...}	10
4	{'gene': 1, 'http://purl.obolibrary.org/obo/NC...}	5
...	...	...
18078	{'http://purl.obolibrary.org/obo/PR_Q2G2S0': 1...}	7
18079	{'gene': 2, 'http://purl.obolibrary.org/obo/PR...}	8
18080	{'PRO:DNx': 5}	5
18081	{'http://www.w3.org/1999/02/22-rdf-syntax-ns#n...}	7
18082	{'Glce': 5, 'EcoGene:EG13561': 1}	6

[18083 rows x 7 columns]

In [89]: **import** re

```
def replace_protein_sample_type(ttl_text):
    # regex to match lines like: PR:000000453_1 a PR:000000453,
    pattern = r'(PR:\d+_\d+)\s+a\s+PR:(\d+),'

    def repl(match):
        sample_iri = match.group(1)
        protein_id = match.group(2)
        # replace with PR:<sample> a <https://proconsortium.org/cgi-bin/entr
        return f'{sample_iri} a <https://proconsortium.org/cgi-bin/entry_pro

    replaced_text = re.sub(pattern, repl, ttl_text)
    return replaced_text

# Example usage:
with open('Pro_Go_vae_file.ttl', 'r') as f:
```

```
ttl_content = f.read()

updated_ttl_content = replace_protein_sample_type(ttl_content)

with open('VAE_results.ttl', 'w') as f:
    f.write(updated_ttl_content)
```